



Universitatea
Transilvania
din Braşov

UNIVERSITATEA TRANSILVANIA DIN BRAŞOV
Facultatea de Inginerie Electrică şi Ştiinţa Calculatoarelor
Programul de studii **Calculatoare** — Anul III

RAPORT DE PRACTICĂ

Proiectarea şi verificarea unui
System-on-Chip RV32I

*Procesor, controler de întreruperi şi integrarea
pe magistrală AXI4 a unui sistem complet*

Student

Bunea Cosmin-Andrei

Instituţia gazdă

Siemens SRL — Electronic Design Romania
AFI Park 1, Bd. 15 Noiembrie nr. 78, Braşov

Coordonator din partea
instituţiei gazdă

Dávid Borzási

Director de departament

Ş.l. dr. ing. STANCA Aurel-Cornel

Perioada stagiului

01.07.2026 — 30.09.2026

Anul universitar

2025/2026

Despre acest raport

Raportul descrie ce am proiectat, ce am integrat și ce am verificat până acum în stagiul de la Siemens, în programul *Digital IC Design & Advanced Verification*. Stagiul este în curs și se încheie la 30 septembrie 2026.

Protocolul AXI4, în măsura în care acest sistem îl folosește, și instrumentele de lucru sunt în Anexa A.

Cifrele din capitolul de verificare sunt citite din jurnalul regresiei rulate pe versiunea curentă a proiectului.

Cuprins

Despre acest raport	ii
1 Contextul stagiului	1
1.1 Instituția gazdă	1
1.2 Obiectivul stagiului	1
1.3 Echipa și repartizarea temelor	1
1.4 Interfața dintre blocuri, stabilită de la început	2
1.5 Etapele lucrării	2
2 Arhitectura sistemului	4
2.1 Calea de date	4
2.2 Harta de adrese	5
2.3 Harta întreruperilor	5
2.4 Blocurile vecine, ca mediu de lucru	6
3 Procesorul RV32I	7
3.1 Modelul de execuție	7
3.2 Structura blocului	8
3.3 Fetch-ul instrucțiunilor și predicția branch-urilor	8
3.4 Controlul pipeline-ului	9
3.4.1 Registrele dintre stagii și bubble-urile	9
3.4.2 Unitatea de hazarduri	9
3.4.3 Ordinea deciziilor în stagiul doi	10
3.4.4 Ce costă fiecare caz	11
3.4.5 Forwarding-ul operanzilor	11
3.5 Unitatea load/store și porturile AXI4-Lite	11
3.6 Registrele de control, trap-urile și întreruperile	13
3.6.1 Urmărirea nivelurilor de trap	13
3.6.2 Cauzele de trap	14
4 Temporizatorul de sistem	15
4.1 Contorul și comparația	15
4.2 Armarea fără declanșare falsă	15
4.3 Citirea contorului din software pe 32 de biți	15
5 Controlerul de întreruperi	16
5.1 De ce are nevoie sistemul de un asemenea bloc	16
5.2 Structura internă	16
5.3 Cum este aleasă o cerere	16
5.4 Ciclul de viață al unei surse	17
5.4.1 Trei mecanisme care nu sunt evidente	17
5.5 Harta de registre	18
6 Integrarea într-un System-on-Chip complet	19
6.1 De ce nu ajungea o magistrală obișnuită	19
6.1.1 Calea de citire	19
6.1.2 Calea de scriere și răspunsul persistent	20
6.2 Decodorul de adrese și bucla combinațională	20
6.3 Arbitrul	21

6.4	Memoriile și nivelul de vârf	22
6.5	Delimitarea față de blocurile colegilor	22
6.6	Traseul complet, pas cu pas	22
7	Verificarea	24
7.1	Strategia	24
7.2	Mediul de verificare	24
7.3	Regresia	25
7.3.1	Parcursul temporizărilor de magistrală	26
7.4	Aserțiunile	26
7.5	Injectarea deliberată de defecte	26
7.6	Proprietăți pe care regresia le fixează explicit	27
7.6.1	Compunerea celor două măști de întreruperi	28
7.7	Rezultate măsurate	28
8	Rezultate și concluzii	30
8.1	Ce funcționează	30
8.2	Ce am învățat	30
8.2.1	Despre proiectare	30
8.2.2	Despre verificare	31
8.2.3	Despre lucrul în echipă și documentație	31
8.3	Concluzie	31
A	Tehnologii, instrumente și protocoale	32
A.1	Limbaje și instrumente	32
A.2	Magistrala AXI4-Lite	32
A.2.1	Canale	33
A.2.2	Handshake-ul	33
A.2.3	Codurile de răspuns	34
A.3	Ce adaugă AXI4-Full	34
A.4	Arhitectura RISC-V RV32I	34
B	Termeni și abrevieri	35
	Bibliografie	36

Listă de figuri

2.1	Calea de date a sistemului. Cutiile din banda mijlocie sunt blocurile pe care le-am scris pentru integrare; cele din banda de sus și de jos sunt blocurile de proiectare, ale mele și ale colegilor. Cele două trasee care se încrucișează — portul A al memoriei duale, alimentat dinspre stânga, și prima ieșire a decodorului DMA, care merge spre dreapta — nu pot fi desenate fără o încrucișare; ea este marcată cu × și se află în spațiu liber.	4
2.2	Traseul întreruperilor. Controlerul primește 16 surse hardware, alege una și o oferă nucleului; nucleul confirmă preluarea și, la sfârșitul tratării, anunță terminarea. . .	6
3.1	Structura procesorului. Fiecare cutie este un modul separat, cu excepția muxului de forwarding, care este logică inline la nivelul de vârf. Am ținut modulele mici și cu o singură responsabilitate, tocmai ca fiecare să poată fi verificat separat — ALU-ul și predictorul au testbench-uri proprii.	8
3.2	Comportamentul slotului de citire. Niciunul dintre cele trei nume nu există ca registru de stare în circuit: ele sunt produse de trei indicatoare independente. Contururile duble marchează starea de după reset.	9
3.3	Ocuparea stagiilor, ciclu cu ciclu. În regim normal o instrucțiune intră în S2 în fiecare ciclu. La o predicție greșită, instrucțiunea de pe drumul greșit este aruncată, S2 execută un bubble, iar instrucțiunea corectă ajunge în S2 la două cicluri după branch — un singur ciclu pierdut.	11
3.4	Mașina de stări a unității load/store. Cele două bucle pe loc nu sunt decorative: ele sunt cazul obișnuit, pentru că un slave poate întârzia oricât.	12
3.5	O încărcare pe magistrala de date, cu un slave care răspunde în două cicluri. Disponibilitatea pe canalul de date se ridică abia după ce adresa a fost acceptată, iar stagiul doi rămâne blocat până când cuvântul este preluat — nu până când sosește.	12
3.6	O stocare unde slave-ul acceptă adresa înaintea datelor. Acesta este cazul pentru care există cele trei indicatoare: disponibilitatea pe canalul de răspuns se ridică abia după ce <i>amândouă</i> handshake-urile, de adresă și de date, s-au încheiat.	13
3.7	Urmărirea trap-urilor în procesor. Nivelurile deschise formează o stivă de biți de fel, nu o pereche de indicatoare: fiecare trap pune un bit, fiecare revenire scoate unul. Astfel, o excepție apărută într-o rutină de întrerupere se întoarce fără să anunțe terminarea întreruperii, iar fiecare nivel imbricat își trimite propriul semnal de terminare.	14
4.1	Secvența corectă de armare. Cât timp jumătatea superioară este încă numai de unu, comparația nu poate trece, deci scrierea jumătății inferioare mai întâi este sigură. În ordinea inversă există o fereastră între cele două scrieri în care o jumătate veche poate declanșa o întrerupere falsă.	15
5.1	Structura internă a controlerului de întreruperi. Rezolvitorul este în întregime combinațional; singurul etaj înregistrat pe calea cererii este perechea de ieșire. Am ales asta ca latența de la apariția unei surse la oferta către procesor să fie de exact un ciclu, indiferent câte surse concurează.	16
5.2	Ciclul de viață al unei surse. Escaladarea modifică doar banda efectivă folosită la construirea cheii de prioritate și este anulată la confirmare, nu la terminare — din momentul în care sursa a fost preluată, nu mai are sens să fie grăbită.	17
6.1	Mașina de stări a căii de citire din puncte. Starea de eroare există ca să răspundă la o rafală nesuportată <i>fără</i> să emită nimic pe magistrală.	19

6.2	Mașina de stări a căii de scriere. Starea finală există exact pentru că protocolul cere un singur răspuns pe rafală: el nu poate fi trimis până nu s-a încheiat ultima tranzacție Lite.	20
6.3	Cele două căi de rutare din decodor. Faza de adresă urmează adresa vie; faza de răspuns urmează exclusiv selecția memorată la handshake-ul de adresă.	21
6.4	Arbitrul. O acordare durează exact o tranzacție și este eliberată pe beat-ul de răspuns, nu la handshake-ul de adresă.	21
A.1	Două transferuri pe același canal. Primul (sarcina utilă A) se încheie imediat, pentru că ambele părți sunt gata în același ciclu. Al doilea (sarcina B) este ținut de sursă încă două cicluri, cât timp destinația nu este gata; sarcina utilă nu se schimbă pe durata așteptării.	33

Capitolul 1

Contextul stagiului

1.1 Instituția gazdă

Am efectuat stagiul de practică la **Siemens SRL**, în departamentul *Electronic Design Romania*, cu sediul în AFI Park 1, Bulevardul 15 Noiembrie nr. 78, Brașov. Coordonator din partea instituției gazdă a fost domnul **Dávid Borzási**.

Siemens face parte dintr-un grup tehnologic prezent global, cu activități în industrie, infrastructură, mobilitate și sănătate. Divizia relevantă pentru stagiul este Siemens EDA, care dezvoltă instrumente de proiectare și verificare pentru circuite ASIC, FPGA și SoC: simulare și verificare funcțională sau formală, verificare fizică și a fiabilității, simulare analogică, precum și soluții embedded construite pe arhitectura RISC-V, destinate aplicațiilor auto, industriale, aerospațiale și medicale. Stagiul s-a desfășurat exact pe acest teren — proiectare RTL și verificare, cu instrumentele și convențiile folosite industrial.

1.2 Obiectivul stagiului

Tema stagiului — *Digital IC Design & Advanced Verification* — a fost proiectarea de la zero a unui System-on-Chip funcțional și verificarea lui. Punctul de plecare, stabilit în ședința de deschidere din 1 iulie 2026, a fost modelul clasic al unui SoC: un nucleu de calcul, o magistrală care îl leagă de periferice și memorii, și un motor de transfer care mută date în fundal, fără să consume timp de procesor.

Fluxul de date țintă a fost enunțat de la început în cinci pași, iar el a rămas criteriul de acceptare al întregului proiect:

1. procesorul configurează motorul DMA prin registre, peste AXI4-Lite;
2. DMA-ul citește și scrie date folosind propria interfață de master AXI4;
3. memoria SRAM dual-port servește ca tampon partajat între procesor și DMA;
4. DMA-ul semnalează terminarea transferului printr-o întrerupere;
5. procesorul prelucrează datele noi.

Obiectivul nu a fost, așadar, un bloc izolat care trece un testbench, ci un sistem în care software rulând pe procesorul propriu programează un periferic propriu, adoarme, este trezit de o întrerupere reală și verifică rezultatul. Acest scenariu rulează astăzi ca test de sistem și este descris în Capitolul 7.

1.3 Echipa și repartizarea temelor

Am lucrat în echipă de trei stagiaari. Împărțirea a fost făcută pe blocuri, cu un document de cerințe separat pentru fiecare, astfel încât cele trei blocuri să poată fi dezvoltate în paralel și integrate la sfârșit.

Tabela 1.1: Repartizarea temelor și responsabilitatea fiecărui bloc

Coleg	Bloc	Conținut
Gabriel	Memorie SRAM dual-port	Două porturi AXI4-Lite independente, 1 KB, detecție și arbitrare de coliziuni, registre de control și de monitorizare a lățimii de bandă.
Andrei	Motor DMA multi-canal	Patru canale independente, scatter-gather pe lanțuri de descriptori, master AXI4-Full cu rafale, arbitru de prioritate, limitare de bandă per canal.
Bunea Cosmin-Andrei	Procesor RV32I, controler de întreruperi, temporizator	Pipeline pe trei stagii, două porturi master AXI4-Lite, controler de întreruperi cu 16 surse hardware și 16 software, temporizator de sistem pe 64 de biți.

Pe lângă blocul propriu mi-am asumat și **integrarea celor trei blocuri într-un SoC funcțional**. Aceasta a însemnat scrierea magistralei care le leagă — decodor de adrese, arbitru, punte de protocol, memorii AXI4-Lite —, a nivelului de vârf al sistemului și a verificării la nivel de sistem. Este partea descrisă în Capitolul 6 și, ca volum de muncă, a fost comparabilă cu proiectarea procesorului.

De ce a contat separarea pe ramuri. Fiecare bloc s-a dezvoltat pe ramura lui, cu testbench-urile lui, fără să depindă de starea celorlalte două. La integrare, cele trei ramuri au fost aduse împreună fără să fie rescrise: nicio ramură de bloc nu a fost rebazată sau suprascrisă. Consecința practică este că fiecare coleg a putut continua să lucreze la blocul lui în timp ce integrarea era în curs, iar o greșeală de integrare nu putea să corupă munca nimănui.

1.4 Interfața dintre blocuri, stabilită de la început

Cele trei blocuri nu s-ar fi putut integra dacă nu ne-am fi înțeles din prima săptămână asupra a trei lucruri. Odată fixate, nu s-au mai schimbat:

- **Un singur protocol de magistrală.** Toate porturile de tip slave sunt AXI4-Lite pe 32 de biți. Singura excepție admisă a fost portul de master al DMA-ului, care este AXI4-Full, pentru că un motor de transfer își pierde rostul dacă nu poate emite rafale. Această excepție este motivul pentru care sistemul are nevoie de o punte de protocol, subiectul Secțiunii 6.1.
- **Un singur domeniu de ceas și un singur reset,** asincron și activ pe 0. Nicio traversare între domenii de ceas nu apare în sistem, ceea ce elimină o clasă întreagă de defecte care oricum nu ar fi putut fi verificată în timpul disponibil.
- **Întreruperile trec toate prin controlerul de întreruperi.** Niciun periferic nu se leagă direct la procesor; fiecare își ridică linia proprie, iar controlerul decide care ajunge la nucleu și în ce ordine.

1.5 Etapele lucrării

Stagiul se desfășoară între 1 iulie și 30 septembrie 2026, în regim hibrid: lucru individual pe parcursul săptămânii și o întâlnire săptămânală cu coordonatorul, la sediu. Întâlnirea are de fiecare dată aceeași structură — ce am făcut de la ultima discuție, cum arată progresul față de ce era planificat, ce urmează, și feedback pe deciziile luate între timp. Acest ritm este și motivul

pentru care etapele de mai jos se citesc ca o secvență și nu ca o listă de sarcini paralele: fiecare a fost discutată și acceptată înainte să înceapă următoarea.

Etapele parcurse până la data acestui raport sunt, în ordine:

Tabela 1.2: Etapele lucrării proprii, în ordine cronologică

Perioadă	Ce am făcut
Iulie, prima jumătate	Arhitectura procesorului: datapath modular, registre între stagii, decodor, generator de imediate, ALU, logica de hazarduri, predictor de branch, unitatea load/store și cele două interfețe AXI4-Lite.
Iulie, a doua jumătate	Controlerul de întreruperi și temporizatorul, registrele de control și stare, instrucțiunea de așteptare WFI , stiva adreselor de retur, contoarele hardware de performanță. Prima infrastructură de verificare: testbench-uri auto-verificatoare, monitoare de protocol, aserțiuni, integrare continuă.
August	Reproiectarea controlerului de întreruperi pentru 16 surse hardware și 16 software, cu benzi de prioritate, preempție și nesting. Eliminarea avertismentelor de compilare, reset determinist pe fiecare bistabil, extinderea regresiei cu teste de bloc separate. Redactarea documentației tehnice.
Sfârșit de august — 1 septembrie	Integrarea: organizarea depozitului pe blocuri, magistrala AXI4-Lite, puntea AXI4-Full → AXI4-Lite, nivelul de vârf, testele de sistem și de stres, revizuirea adversarială a propriei lucrări și fixarea prin teste a proprietăților pe care le-a stabilit.

Capitolul 2

Arhitectura sistemului

2.1 Calea de date

Sistemul are trei masteri — portul de instrucțiuni al procesorului, portul lui de date, și portul de master al DMA-ului — și șase slave-uri. Între ei stă o magistrală pe care am scris-o eu în întregime: trei decodoare de adrese, un arbitru, o punte de protocol și două memorii, adică șapte instanțe ale celor patru module descrise în Capitolul 6.

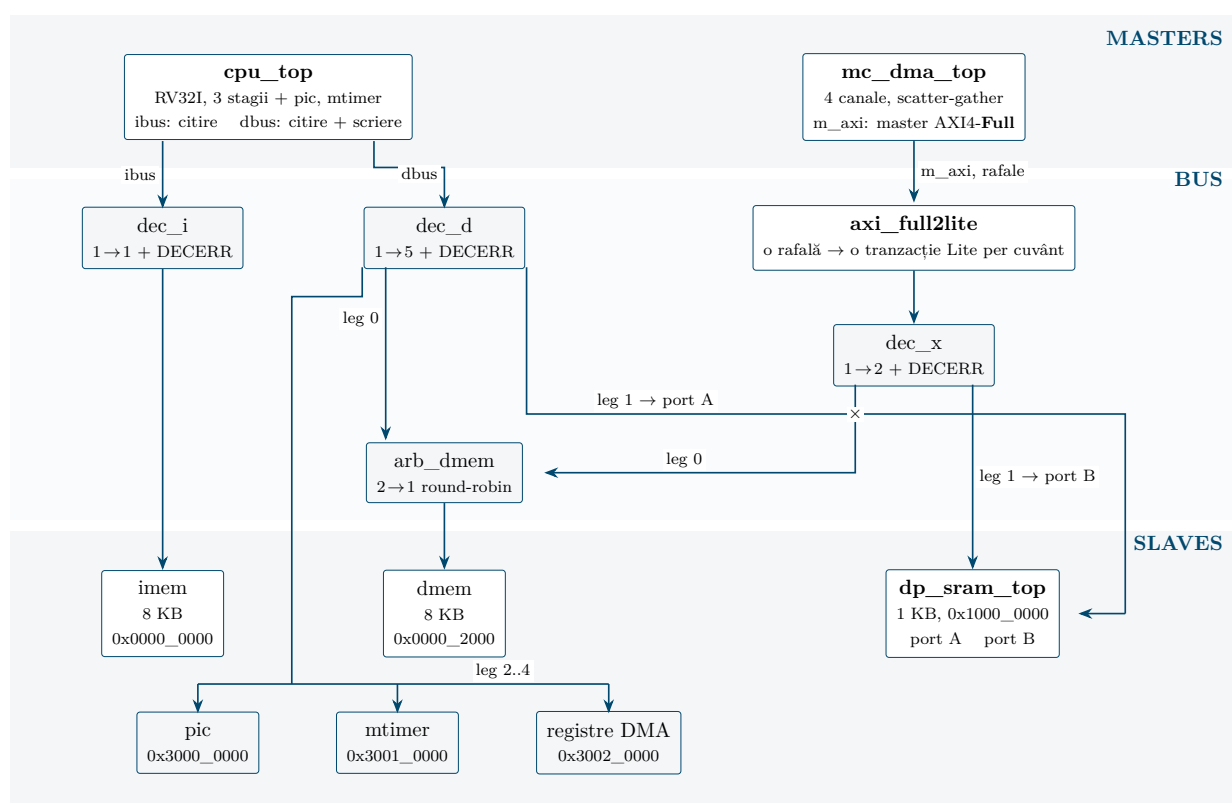


Figura 2.1: Calea de date a sistemului. Cutiile din banda mijlocie sunt blocurile pe care le-am scris pentru integrare; cele din banda de sus și de jos sunt blocurile de proiectare, ale mele și ale colegilor. Cele două trasee care se încrucișează — portul A al memoriei duale, alimentat dinspre stânga, și prima ieșire a decodului DMA, care merge spre dreapta — nu pot fi desenate fără o încrucișare; ea este marcată cu × și se află în spațiu liber.

Două trasee prin magistrală, urmărite de la un capăt la altul:

- **O încărcare din memoria duală, făcută de procesor.** Portul de date intră în **dec_d**, iese pe ieșirea 1 și ajunge direct la portul A al memoriei. Nu există arbitru pe acest traseu: portul A îi aparține procesorului în întregime.
- **DMA-ul mutând 32 de octeți din memoria de date.** Portul de master emite o singură rafală INCR de opt cuvinte. Puntea o desface în opt tranzacții AXI4-Lite separate, care intră în **dec_x**, ies pe ieșirea 0 și ajung la arbitru, unde fiecare se așază la coadă în spatele cererilor procesorului.

2.2 Harta de adrese

Harta este definită într-un singur loc, iar decodoarele o primesc ca parametru. Trei proprietăți ale ei sunt decizii, nu valori implicite, și fiecare are un test dedicat care cade dacă proprietatea se pierde.

Tabela 2.1: Harta de adrese și accesibilitatea. O celulă goală înseamnă că masterul respectiv nu are nicio cale către acel slave; un acces acolo primește DECERR.

Slave	Bază	Dim.	ibus	dbus	DMA
imem	0x0000_0000	8 KB	✓	—	—
dmem	0x0000_2000	8 KB	—	✓	✓
dp_sram	0x1000_0000	1 KB	—	✓(A)	✓(B)
pic	0x3000_0000	256 B	—	✓	—
mtimer	0x3001_0000	256 B	—	✓	—
registre DMA	0x3002_0000	256 B	—	✓	—

Fiecare fereastră are exact dimensiunea blocului ei. Masca de decodare a unei ferestre este exact mărimea blocului, nu mărimea intervalului dintre blocuri. Perifericele sunt așezate din 64 în 64 de kiloocteți, dar fiecare are o fereastră de numai 256 de octeți, fiindcă atât are banca lui de registre. Dacă fereastra ar fi de 64 KB, adresa 0x3000_0100 ar cădea tot în controlerul de întreruperi și ar ateriza pe primul lui registru, răspunzând *OKAY* — adică exact eșecul pe care decodorul există ca să-l prevină. Cu fereastra de 256 de octeți, aceeași adresă nu nimerește nicio fereastră și primește DECERR.

Accesibilitatea nu este uniformă, și asta este intenționat. Portul de instrucțiuni ajunge numai la memoria de instrucțiuni, deci un contor de program scăpat de sub control primește o eroare de decodare în loc să execute registrele unui periferic. Portul de date ajunge peste tot în afară de memoria de instrucțiuni, deci nu există cale prin care programul să-și rescrie propriul cod. DMA-ul ajunge numai la memoria de date și la memoria duală, deci un descriptor greșit nu poate reprograma controlerul de întreruperi.

Memoria duală este văzută la aceeași adresă din ambele părți. Procesorul o atinge pe portul A, DMA-ul pe portul B, ambii la 0x1000_0000. Acesta este lucrul care ține detectorul de coliziuni al blocului viu în sistemul asamblat: dacă cei doi masteri ar vedea memoria la adrese diferite, nu s-ar putea ciocni niciodată pe același cuvânt.

2.3 Harta întreruperilor

Niciun periferic nu se leagă direct la procesor. Fiecare își ridică linia proprie către controlerul de întreruperi, iar acesta oferă nucleului o singură cerere, pe cea mai urgentă.

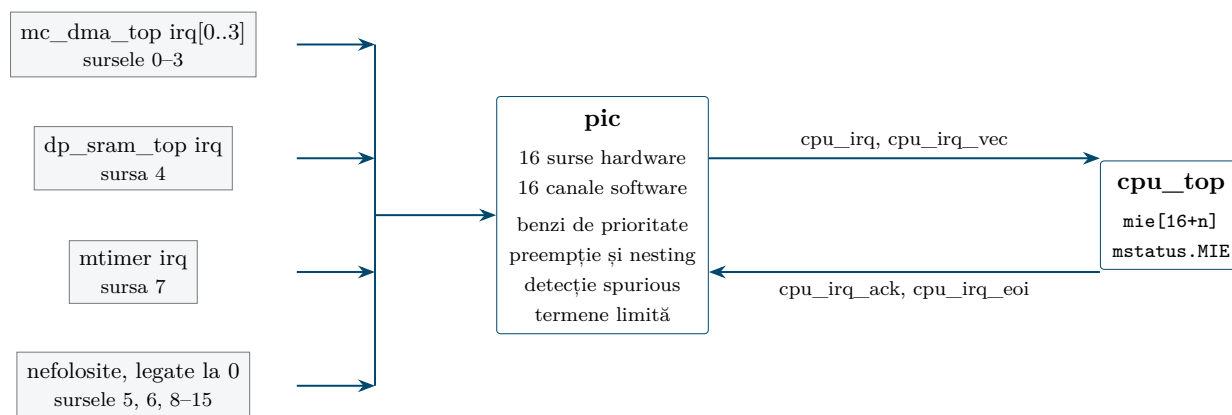


Figura 2.2: Traseul întreruperilor. Controlerul primește 16 surse hardware, alege una și o oferă nucleului; nucleul confirmă preluarea și, la sfârșitul tratării, anunță terminarea.

O întrerupere de la un canal DMA trece prin **trei măști independente** înainte să ajungă la o rutină de tratare: masca proprie a DMA-ului, apoi masca controlerului de întreruperi, apoi bitul din registrul de activare al procesorului, condiționat la rândul lui de bitul global de activare a întreruperilor. Felul în care ultimele două se compun este o decizie de proiectare în sine, descrisă în Secțiunea 7.6.1.

2.4 Blocurile vecine, ca mediu de lucru

Motorul DMA și memoria SRAM dual-port nu sunt lucrarea mea, dar sunt mediul în care blocul meu trebuie să funcționeze și sursa constrângerilor care au dictat forma magistralei.

Tabela 2.2: Proprietățile blocurilor vecine care au constrâns integrarea

Bloc	Ce a impus asupra sistemului
Motorul DMA (4 canale)	Are două interfețe: un port slave AXI4-Lite, prin care procesorul îi programează registrele, și un port master AXI4-Full, prin care mută datele. Portul de master emite rafale INCR de opt cuvinte de 32 de biți. Fiindcă niciun slave din sistem nu înțelege rafale, acest port este singurul motiv pentru care există puntea din Secțiunea 6.1. Fiecare dintre cele patru canale își ridică propria linie de întrerupere la terminarea sau la eroarea unui transfer; ele intră în controlerul meu de întreruperi pe sursele 0-3.
Memoria SRAM dual-port (1 KB)	Are două porturi AXI4-Lite fizic independente. Aceasta este proprietatea care i-a scutit de arbitru: procesorul primește portul A, DMA-ul portul B, și niciunul nu stă la coadă după celălalt. Din întreaga memorie de 1 KB, primele opt cuvinte sunt registrele ei de control, iar restul este matricea de date. Blocul are un detector de coliziuni propriu: dacă ambele porturi ating același cuvânt în același ciclu și cel puțin unul scrie, scriitorul câștigă și cititorul primește SLVERR. Peste un prag configurabil de coliziuni, ambele porturi sunt oprite pentru un număr de cicluri. Blocul își ridică propria linie de întrerupere, care intră în controlerul meu pe sursa 4.

Cele două proprietăți îngroșate sunt exact sursa a ceea ce am scris: rafalele au impus puntea de protocol, iar cele două porturi au impus ca memoria să fie văzută la aceeași adresă din ambele părți.

Capitolul 3

Procesorul RV32I

Procesorul este blocul cu care am început stagiul și cel la care am lucrat cel mai mult.

3.1 Modelul de execuție

Am proiectat nucleul cu un pipeline pe trei stagii. Numărul de stagii a fost fixat de tema de proiectare; *repartizarea muncii pe stagii* a fost însă decizia mea, și ea se resimte în tot restul implementării.

Tabela 3.1: Stagiile pipeline-ului și munca făcută în fiecare

Stagiu	Nume	Ce se face
S1	Fetch	Generarea contorului de program, citirea instrucțiunii pe magistrala de instrucțiuni, consultarea predictorului de branch.
S2	Decode + Execute	Decode, citirea registrelor, ALU, rezolvarea branch-ului, accesul la registrele de control și stare, <i>întreaga</i> tranzacție de memorie, și toate deciziile de excepție și de întrerupere.
S3	Writeback	Scrierea rezultatului în fișierul de registre.

Punând decodarea, execuția și accesul la memorie în același stagiu, o instrucțiune fie se termină complet, fie nu începe deloc. Aceasta este exact proprietatea care face trap-urile **precise** fără niciun mecanism de recuperare: nu există niciodată o instrucțiune pe jumătate executată pe care un trap să o repare.

Costul este un drum combinațional lung — decode → ALU → calcul de adresă → verificare de aliniere — care ar limita frecvența de ceas pe siliciu real. Pentru ținta acestui proiect, care este simularea comportamentală, costul nu se plătește, deci am făcut schimbul deliberat.

3.2 Structura blocului

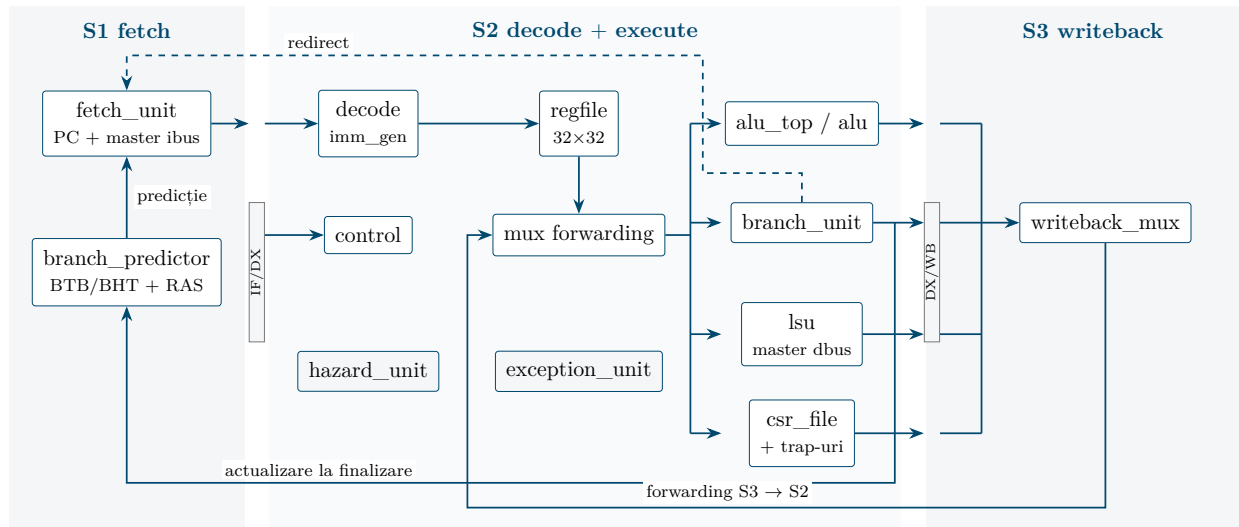


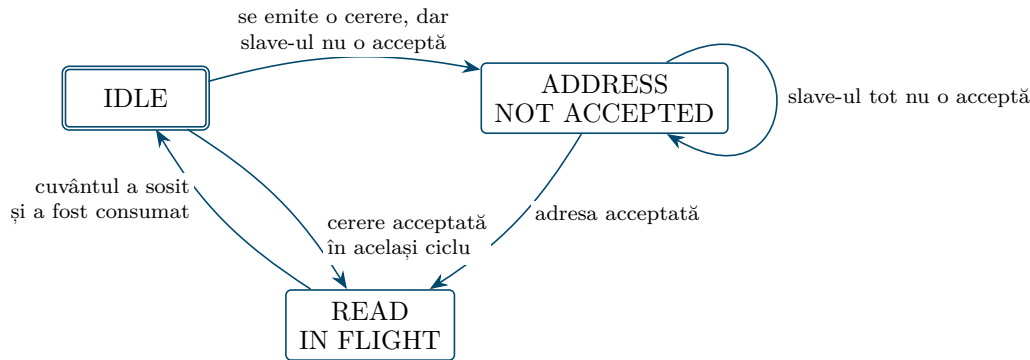
Figura 3.1: Structura procesorului. Fiecare cutie este un modul separat, cu excepția muxului de forwarding, care este logică inline la nivelul de vârf. Am ținut modulele mici și cu o singură responsabilitate, tocmai ca fiecare să poată fi verificat separat — ALU-ul și predictorul au testbench-uri proprii.

Portul extern este o intrare de ceas, un reset, cele două porturi master AXI4-Lite și cele șase semnale ale interfeței cu controlerul de întreruperi — cererea, numărul sursei oferite, setul de surse în așteptare, masca procesorului, confirmarea și terminarea — plus un semnal de observabilitate care spune dacă un trap este deschis. Instrucțiunea de așteptare, stiva adreselor de retur și contoarele de performanță se văd toate prin această listă, fără niciun semnal propriu. Este o constrângere pe care mi-am impus-o deliberat: procesorul este un bloc între altele, într-un sistem construit în paralel de alți doi oameni, iar orice semnal adăugat la interfață ar fi cerut o modificare la ei.

3.3 Fetch-ul instrucțiunilor și predicția branch-urilor

Unitatea de fetch este master pe magistrala de instrucțiuni și menține o singură tranzacție în curs. Am reprezentat-o prin trei semnale de stare, nu printr-un registru de stare: „cererea de adresă este emisă și neacceptată”, „citirea este în curs” și „instrucțiunea care vine trebuie aruncată”.

Al treilea vine dintr-o restricție a protocolului. O citire AXI **nu poate fi anulată** după ce semnalul de validare al adresei a fost ridicat. Când predictorul greșește și pipeline-ul trebuie redirectionat, cererea deja emisă nu poate fi retrasă — singura soluție corectă este să lași răspunsul să vină și să-l ignori. De aceea semnalul de aruncare este un indicator ortogonal, nu o a patra stare: poate fi armat din oricare dintre cele două stări ocupate.



Indicatorul de aruncare este ortogonal celor trei stări.

Se aruncează la un redirect venit în oricare dintre stările ocupate și se stinge când cuvântul greșit este înghițit. Nu poate fi o a patra stare, fiindcă citirea nu poate fi oprită — doar rezultatul ei poate fi ignorat.

Figura 3.2: Comportamentul slotului de citire. Niciunul dintre cele trei nume nu există ca registru de stare în circuit: ele sunt produse de trei indicatoare independente. Contururile duble marchează starea de după reset.

Predictorul folosește un tabel de 128 de intrări care combină ținta branch-ului cu un contor de direcție, plus o stivă separată de 8 adrese pentru revenirile din funcții. Stiva există pentru că un return are aproape întotdeauna o altă țintă decât data trecută, deci tabelul obișnuit nu îl poate prezice; adresa de retur, în schimb, este cunoscută exact în momentul apelului.

3.4 Controlul pipeline-ului

3.4.1 Registrele dintre stagii și bubble-urile

Ambele registre dintre stagii — IF/DX și DX/WB — sunt construite la fel: un bit de validitate plus sarcina utilă. Bitul de validitate este ceea ce face o bubble sigur: nimic în aval nu se uită la sarcina utilă dacă bitul este zero, deci un bubble înseamnă pur și simplu *valid* = 0, iar un flush produce o instrucțiune nulă garantată, indiferent ce conțin biții de date.

Alternativa ar fi fost injectarea codului unei instrucțiuni care nu face nimic. Acea ar fi cerut ca traseul de flush să producă 32 de biți corecți și ca fiecare decodor din aval să îi interpreteze — mai mult hardware și mai multe feluri de a greși. Sarcina utilă se încarcă doar când o instrucțiune reală avansează, deci bubble-urile și stall-urile lasă bistabilele liniștite.

3.4.2 Unitatea de hazarduri

Toate deciziile de stall, de flush și de forwarding al operanzilor se iau într-un singur modul. Niciun stagiou nu își gestionează singur avansul, iar controlul întreg încapă în câteva ecuații combinaționale. Consecința practică este că nu există nicio mașină de stări care să conducă pipeline-ul — ceea ce înseamnă și că nu există stări nevalide în care el să ajungă.

Tabela 3.2: Sursele de stall și de flush

Categorie	Surse
Stall (două)	O tranzacție de date în curs pe magistrală, și o instrucțiune de așteptare adormită. Atât. Un fetch de instrucțiune care nu este gata <i>nu</i> creează un stall în stagiul doi: produce un bubble, adică stagiul avansează, doar că fără instrucțiune.
Flush (trei)	Predicție greșită rezolvată în S2, intrare în trap, și revenire din trap. Toate trei fac același lucru — ridică semnalul de redirect și încarcă un bubble în registrul dintre stagii — deci există un singur mecanism, nu trei.

Hazardul clasic „încărcare urmată de folosire” nu există aici. Într-un pipeline cu trei stagii, datele unei încărcări sunt deja în S3 în momentul în care instrucțiunea care le consumă intră în S2, deci o singură cale de forwarding $S3 \rightarrow S2$ le acoperă, fără niciun ciclu pierdut.

Când stall-ul și flush-ul cer lucruri diferite, flush-ul câștigă, fiindcă tocmai invalidează instrucțiunea pe care stall-ul o aștepta. Regula are însă o a doua jumătate, care vine din același semnal: fiecare condiție de flush este condiționată de avansul stagiului doi, deci un trap *nu poate* părăsi S2 cât timp o tranzacție de date este încă în curs. Magistrala nu rămâne niciodată cu o tranzacție orfană, al cărei răspuns nu îl mai așteaptă nimeni.

3.4.3 Ordinea deciziilor în stagiul doi

Toate deciziile arhitecturale se iau în S2, într-o ordine fixă:

întrerupere > excepție sincronă > revenire din trap > predicție greșită

Întreruperea este testată prima, și anume *înainte* ca instrucțiunea să emită vreo tranzacție de date, ca instrucțiunea întreruptă să poată fi reluată identic după ce rutina de tratare se întoarce. O excepție trece înaintea unei predicții greșite pentru că o instrucțiune care a dat fault nu mai are niciun drum corect de urmat.

3.4.4 Ce costă fiecare caz

(a) regim normal — o instrucțiune pe ciclu

	ciclul 1	ciclul 2	ciclul 3	ciclul 4	ciclul 5
S1	i1	i2	i3	i4	i5
S2		i1	i2	i3	i4
S3			i1	i2	i3

(b) branch prezis greșit — costă exact un ciclu

	ciclul 1	ciclul 2	ciclul 3	ciclul 4	ciclul 5
S1	branch	drum greșit	țintă	ț+1	ț+2
S2		branch	bubble	țintă	ț+1
S3			branch	bubble	țintă

S2 rezolvă branch-ul și redirecționează fetch-ul

Figura 3.3: Ocuparea stagiilor, ciclu cu ciclu. În regim normal o instrucțiune intră în S2 în fiecare ciclu. La o predicție greșită, instrucțiunea de pe drumul greșit este aruncată, S2 execută un bubble, iar instrucțiunea corectă ajunge în S2 la două cicluri după branch — un singur ciclu pierdut.

Tabela 3.3: Cele patru cazuri care determină debitul

Caz	Comportament
Operație ALU	O instrucțiune intră în S2 în fiecare ciclu; un ciclu per instrucțiune, măsurat pe cod liniar cu memorie de instrucțiuni cu latență de un ciclu.
Încărcare / stocare	S2 se blochează pe toată durata tranzacției pe magistrală. S1 poate termina un fetch deja pornit și îl parchează; nimic nu intră și nimic nu iese din S2.
Predicție greșită	Costă exact un ciclu, ca în Figura 3.3.
Intrare în trap	Niciun ciclu peste redirect. Instrucțiunea care a dat fault este anulată pe drumul ei către registrul următor, în același ciclu în care starea de trap este scrisă.

3.4.5 Forwarding-ul operanzilor

Unitatea de hazarduri calculează și potrivirile de forwarding: registrul destinație din S3 comparat cu fiecare registru sursă din S2, cu registrul zero exclus. O potrivire înlocuiește valoarea citită din fișierul de registre cu valoarea care se scrie.

3.5 Unitatea load/store și porturile AXI4-Lite

Unitatea load/store conține singura mașină de stări codificată explicit în procesor. Am ținut-o deliberat minimală — trei stări — pentru că restul complexității stă în urmărirea handshake-urilor, nu în stare.

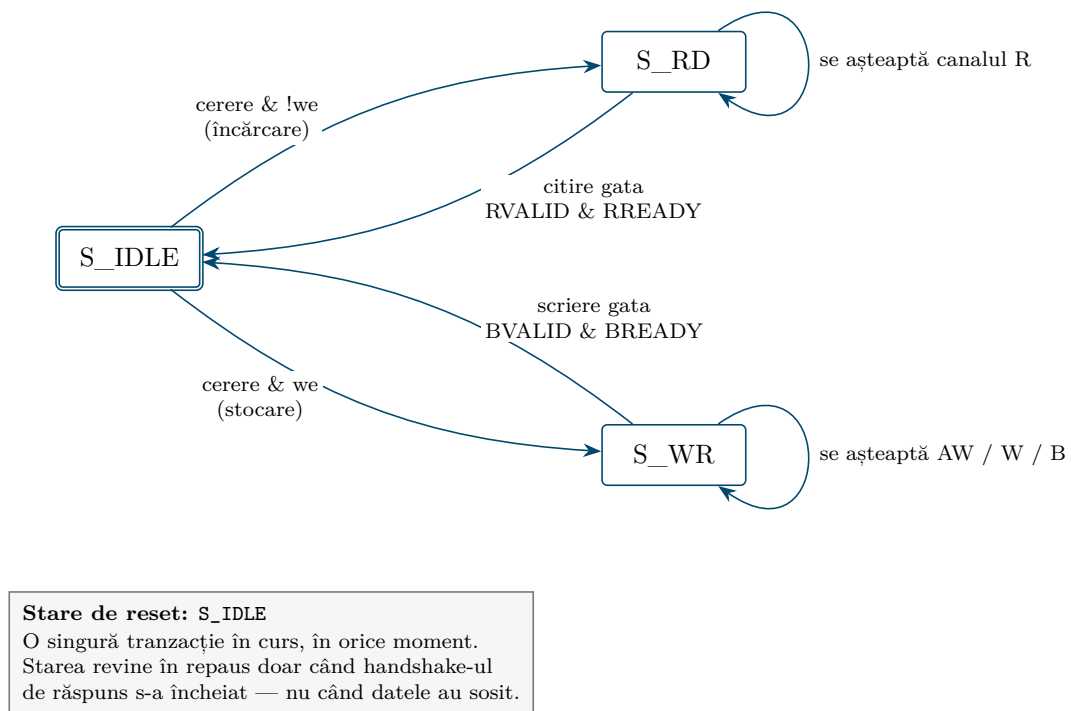


Figura 3.4: Mașina de stări a unității load/store. Cele două bucle pe loc nu sunt decorative: ele sunt cazul obișnuit, pentru că un slave poate întârzia oricât.

Starea singură nu spune cât de departe a ajuns o tranzacție, pentru că o scriere are **trei canale independente**: adresa, datele și răspunsul. Adresa poate fi acceptată înaintea datelor sau invers. De aceea am adăugat trei indicatoare separate, câte unul per canal, iar starea revine în repaus doar când toate trei s-au închis. Aceasta a fost lecția practică cea mai directă despre AXI: nu există un singur semnal „gata”, iar a presupune o ordine între canale este exact felul în care un master pare corect până când întâlnește un slave mai lent.

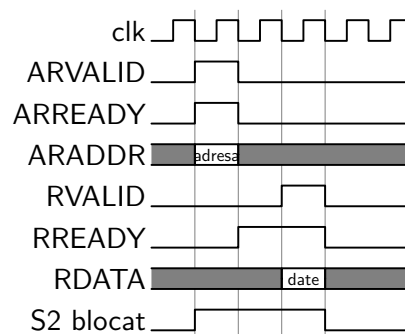


Figura 3.5: O încărcare pe magistrala de date, cu un slave care răspunde în două cicluri. Disponibilitatea pe canalul de date se ridică abia după ce adresa a fost acceptată, iar stagiul doi rămâne blocat până când cuvântul este preluat — nu până când sosește.

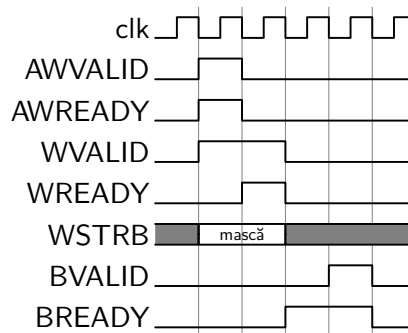


Figura 3.6: O stocare unde slave-ul acceptă adresa înaintea datelor. Acesta este cazul pentru care există cele trei indicatoare: disponibilitatea pe canalul de răspuns se ridică abia după ce *amândouă* handshake-urile, de adresă și de date, s-au încheiat.

Măștile pe octet. O stocare de octet sau de jumătate de cuvânt se face prin măștile `WSTRB`, nu prin citire-modificare-scriere. Unitatea calculează masca din dimensiunea accesului și din cei doi biți de jos ai adresei. Un acces nealiniat nu este emis deloc pe magistrală: este oprit în procesor și produce o excepție precisă, cu adresa care a greșit salvată în registrul dedicat.

3.6 Registrele de control, trap-urile și întreruperile

Am implementat registrele de control și stare în măsura cerută de un sistem cu întreruperi: starea globală de trap, activarea per sursă, vectorul de trap, adresa de retur, cauza, valoarea asociată, identificarea implementării și contoarele de performanță.

3.6.1 Urmărirea nivelurilor de trap

Nucleul ține **câte un bit pentru fiecare nivel de trap deschis**, pe o stivă adâncă de 16 — exact numărul maxim de niveluri pe care controlerul de întreruperi le acceptă. Bitul spune ce fel de trap a deschis nivelul: 1 pentru o întrerupere, 0 pentru o excepție sincronă. Fiecare trap acceptat pune un bit, fiecare revenire scoate unul, iar semnalul de terminare pleacă spre controlerul de întreruperi numai dacă bitul scos este 1.

Un singur indicator „suntem într-o întrerupere”, ridicat la orice întrerupere și coborât la orice revenire, ar părea suficient, dar nu ar putea deosebi două situații:

- **Două rutine imbricate.** Amândouă revin, dar un indicator unic ar produce un singur semnal de terminare în loc de două, iar nivelul exterior ar rămâne deschis pentru totdeauna în stiva controlerului, cu sursa lui blocată.
- **O excepție sincronă în interiorul unei rutine de tratare.** Revenirea din excepție ar trimite semnalul de terminare al întreruperii, închizând-o în timp ce rutina ei încă rulează.

Stiva de biți de fel deosebește ambele cazuri fără logică suplimentară: felul trap-ului care se închide este chiar bitul scos de pe stivă.

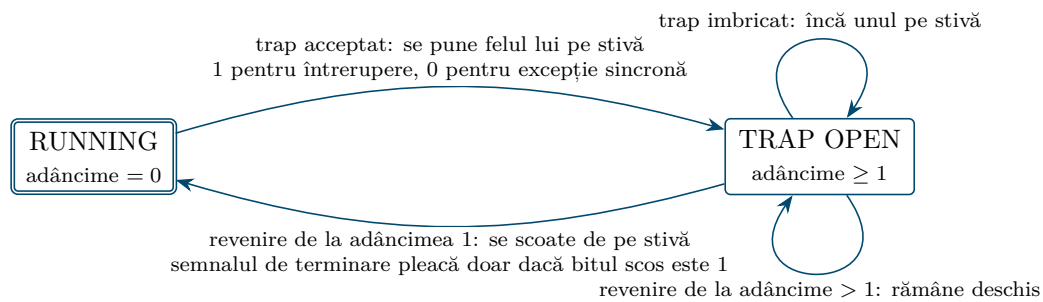


Figura 3.7: Urmărirea trap-urilor în procesor. Nivelurile deschise formează o stivă de biți de fel, nu o pereche de indicatoare: fiecare trap pune un bit, fiecare revenire scoate unul. Astfel, o excepție apărută într-o rutină de întrerupere se întoarce fără să anunțe terminarea întreruperii, iar fiecare nivel imbricat își trimite propriul semnal de terminare.

3.6.2 Cauzele de trap

Cauzele sunt evaluate într-un lanț de prioritate fix, nu în ordinea numerică a codurilor lor. Ordinea contează atunci când două condiții sunt adevărate în același ciclu — de exemplu o instrucțiune ilegală la o adresă nealiniată — și am ales-o astfel încât cauza raportată să fie cea care a apărut prima în execuția instrucțiunii.

Tabela 3.4: Cauzele de trap implementate și valoarea salvată pentru fiecare

Cauză	Valoarea asociată salvată
Întrerupere externă	— (numărul sursei intră în codul cauzei)
Adresă de instrucțiune nealiniată	adresa țintă calculată
Eroare de acces la instrucțiune	adresa care a produs eroarea pe magistrală
Instrucțiune ilegală	cuvântul de instrucțiune în sine
Punct de întrerupere software	adresa instrucțiunii
Adresă de încărcare / stocare nealiniată	adresa efectivă calculată
Eroare de acces la încărcare / stocare	adresa care a primit eroare
Apel de mediu din modul mașină	—

Instrucțiunea de așteptare. Instrucțiunea *WFI* oprește fetch-ul de instrucțiuni până la sosirea unei întreruperi. Am implementat-o astfel încât magistrala de instrucțiuni să tacă efectiv pe durata așteptării — nu doar să ignore rezultatele — pentru că într-un sistem real acesta este singurul mod în care oprirea are vreun sens. Este și mecanismul pe care se sprijină testul de sistem: procesorul programează DMA-ul, execută *WFI*, și este trezit de întreruperea de terminare a transferului.

Capitolul 4

Temporizatorul de sistem

Temporizatorul este cel mai mic dintre blocurile pe care le-am proiectat și, din punctul de vedere al sistemului, este doar încă o sursă de întrerupere — dar este sursa care face posibilă orice formă de programare în timp.

4.1 Contorul și comparația

Implementează perechea clasică din specificația RISC-V: un contor liber de 64 de biți care numără cicluri, și un registru de comparație de 64 de biți. Când contorul atinge sau depășește comparația, linia de întrerupere se ridică și rămâne sus până când software-ul mută comparația mai departe.

Linia de întrerupere intră în controlerul de întreruperi pe sursa 7.

Ambele registre sunt de 64 de biți, dar magistrala este de 32, deci fiecare se accesează în două jumătăți. De aici vin cele două subtilități ale blocului, una la scriere și una la citire.

4.2 Armarea fără declanșare falsă

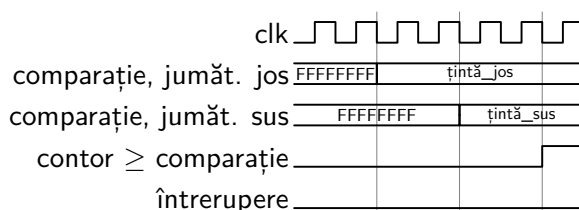


Figura 4.1: Secvența corectă de armare. Cât timp jumătatea superioară este încă numai de unu, comparația nu poate trece, deci scrierea jumătății inferioare mai întâi este sigură. În ordinea inversă există o fereastră între cele două scrieri în care o jumătate veche poate declanșa o întrerupere falsă.

Ordinea sigură este deci: jumătatea inferioară, apoi cea superioară. Am documentat-o ca parte a contractului blocului, nu ca recomandare — este genul de detaliu care, nescris, produce un defect care apare o dată la câteva mii de armări și nu se reproduce niciodată la cerere.

4.3 Citirea contorului din software pe 32 de biți

Aceeași problemă apare simetric la *citirea* contorului: jumătatea inferioară se poate rostogoli între cele două tranzații de magistrală. Soluția uzuală, pe care software-ul de test o folosește, este să citească jumătatea superioară, apoi cea inferioară, apoi din nou cea superioară, și să reia dacă prima și ultima diferă.

Capitolul 5

Controlerul de întreruperi

5.1 De ce are nevoie sistemul de un asemenea bloc

Procesorul primește, la un moment dat, o singură cerere de întrerupere, însoțită de numărul sursei care a produs-o. Sistemul are însă șase surse hardware conectate — patru canale DMA, memoria duală și temporizatorul — plus canalele software. Ceva trebuie să decidă care dintre ele ajunge la nucleu, în ce ordine, și ce se întâmplă când una mai urgentă apare în timp ce alta este deja tratată.

Am proiectat controlerul pentru **16 surse hardware și 16 canale software**, adică 32 de surse logice care intră în același lanț de rezoluție. O cerere software se contopește cu slotul hardware corespunzător, deci odată intrată în lanț este indistinguibilă de una hardware: aceeași prioritate, același termen limită, același comportament la imbricare. Aceasta a fost o decizie conștientă — alternativa, un lanț separat pentru software, ar fi dublat logica de rezoluție fără niciun câștig.

5.2 Structura internă

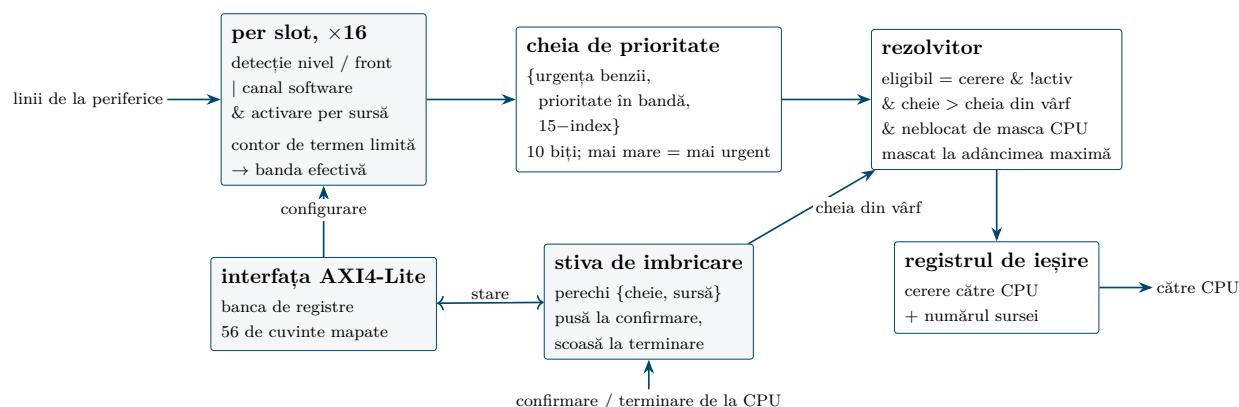


Figura 5.1: Structura internă a controlerului de întreruperi. Rezolvitorul este în întregime combinațional; singurul etaj înregistrat pe calea cererii este perechea de ieșire. Am ales asta ca latența de la apariția unei surse la oferta către procesor să fie de exact un ciclu, indiferent câte surse concurează.

5.3 Cum este aleasă o cerere

Rezoluția se face pe o **cheie de prioritate** de 10 biți, construită din trei câmpuri concatenate. Comparăția între două surse devine astfel o simplă comparație de numere întregi, ceea ce înseamnă un singur comparator în loc de o cascadă de reguli:

Tabela 5.1: Câmpurile cheii de prioritate, de la cel mai semnificativ

Câmp	Biți	Rol
Urgența benzii	2	Sursele sunt grupate în patru benzi; ordinea benzilor este programabilă, deci software-ul poate rearanja urgențele fără să reconfigureze fiecare sursă.
Prioritate în bandă	4	Ordonează sursele din aceeași bandă.
15-index	4	Departajare finală, deterministă: la egalitate perfectă câștigă sursa cu numărul mai mic. Nu există niciodată ambiguitate.

O sursă este eligibilă dacă are o cerere activă, nu este deja în tratare, nu este mascată de procesor, cheia ei este strict mai mare decât cheia din vârful stivei de imbricare, iar adâncimea stivei nu a atins limita. Ultimele două condiții sunt exact ceea ce implementează **preemptia**: o sursă mai urgentă poate întrerupe rutina uneia mai puțin urgentă, dar una la fel de urgentă sau mai puțin urgentă nu.

5.4 Ciclul de viață al unei surse

Controlerul nu are un registru de stare per sursă. Comportamentul de mai jos este produs de câteva vectoare de indicatoare independente — front reținut, cerere software, în tratare, escaladat, spurious. O sursă poate fi deci simultan în așteptare și escaladată.

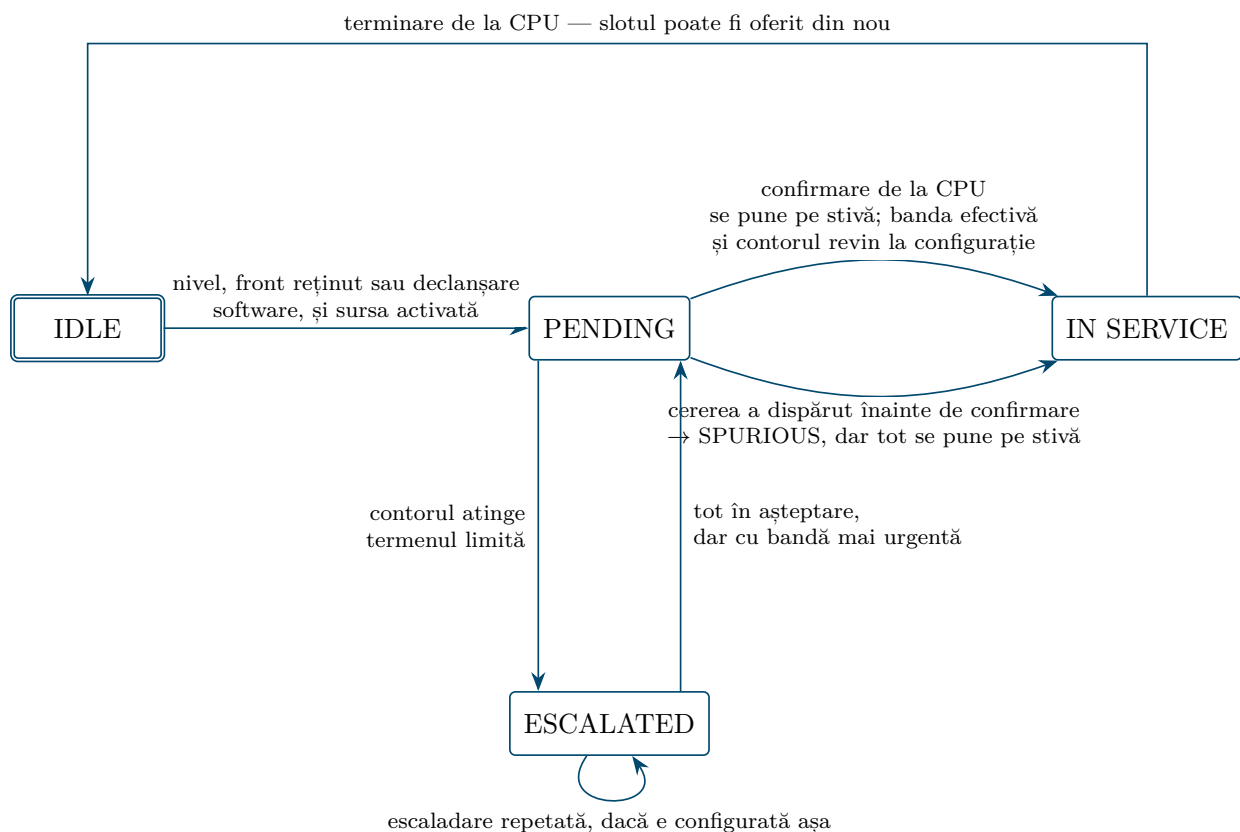


Figura 5.2: Ciclul de viață al unei surse. Escaladarea modifică doar banda efectivă folosită la construirea cheii de prioritate și este anulată la confirmare, nu la terminare — din momentul în care sursa a fost preluată, nu mai are sens să fie grăbită.

5.4.1 Trei mecanisme care nu sunt evidente

Cererile spurious. O sursă declanșată pe nivel poate să dispară între momentul în care controlerul o oferă și momentul în care procesorul o confirmă — perifericul și-a rezolvat singur condiția între timp. Controlerul tot împinge nivelul pe stivă și tot îl marchează, pentru că procesorul a intrat deja în rutina de tratare și *trebuie* să iasă din ea printr-un semnal de terminare. Alternativa — să nu împingi nimic — ar lăsa stiva desincronizată față de nucleu, iar asta este mult mai rău decât o rutină care rulează degeaba o dată.

Termenele limită și escaladarea. Fiecare sursă are un contor care măsoară de cât timp așteaptă neservită. La depășirea unui prag configurabil, banda ei efectivă urcă, deci cheia ei crește și devine mai probabil să fie aleasă. Este un mecanism împotriva înfometării: o sursă cu

prioritate mică nu poate fi ignorată la nesfârșit de un flux constant de surse mai urgente. Contorul este citibil oricând, ceea ce îl face și o măsurătoare de latență per sursă.

Declanșarea software protejată prin cheie. Ridicarea unei cereri software cere scrierea simultană a bitului de cerere și a unei chei fixe în aceeași tranzacție. Ștergerea nu cere cheie. Asimetria este intenționată: cheia apără împotriva creării accidentale a unei întreruperi, nu împotriva înlăturării ei.

5.5 Harta de registre

Tabela 5.2: Harta de registre a controlerului de întreruperi, 56 de cuvinte mapate

Ofset	Registru	Acces	Reset	Rol
0x00--0x3C	configurare surse	RW	0	Tip de declanșare, bandă, prioritate în bandă, termen limită.
0x40--0x7C	declanșare software	RW	0	Cerere software, protejată prin cheie.
0x80--0xBC	stare surse	RO	0	Starea vie a fiecărei surse.
0xC0	configurarea benzilor	RW	0x1B	Urgența fiecărei benzi.
0xC4	starea imbricării	RO	0	Adâncime, sursa din vârf, depășire.
0xC8	adâncime maximă	RW	8	Limita de imbricare.
0xCC	sursa în tratare	RO	0	Cea mai interioară sursă activă.
0xD0	jurnal spurious	R/W1C	0	Jurnal persistent per sursă.
0xD4	politica de escaladare	RW	0	Cum urcă banda la depășirea termenului.
0xD8	activare per sursă	RW	0	Masca principală.
0xDC	stare globală	R/W1C	0	Indicatori persistenți de eveniment.

Două decizii din tabelul de mai sus le-am luat explicit și le-am verificat cu teste dedicate:

Biții rezervați sunt legați hardware la zero. Orice poziție marcată rezervată citește 0 și înghite o scriere: valoarea este mascată *înainte* să ajungă în registru, nu stocată și citită înapoi. Este comportamentul pe care specificația privilegiată RISC-V îl folosește în aceeași situație, și este ceea ce permite unei revizii ulterioare să definească unul dintre acei biți fără să strice software-ul care s-a întâmplat să scrie 1 acolo.

Blocul iese din reset complet dezarmat. Masca principală este zero, deci nimic nu poate ajunge la procesor până când software-ul nu înscrie explicit o sursă. Toate sursele sunt pe nivel, în banda 0, cu prioritate 0 și fără termene limită. Singurele două registre care nu se resetează la zero sunt cele unde zero ar fi greșit: ordinea benzilor are nevoie de o ordonare implicită rezonabilă, iar adâncimea maximă de imbricare de zero ar însemna un controler care nu oferă niciodată nimic.

Capitolul 6

Integrarea într-un System-on-Chip complet

Cele trei blocuri existau, fiecare trecându-și propriile teste, pe ramuri separate. Între ele nu exista nimic. Ca să devină un sistem, am scris patru module noi, un nivel de vârf și o hartă de adrese.

6.1 De ce nu ajungea o magistrală obișnuită

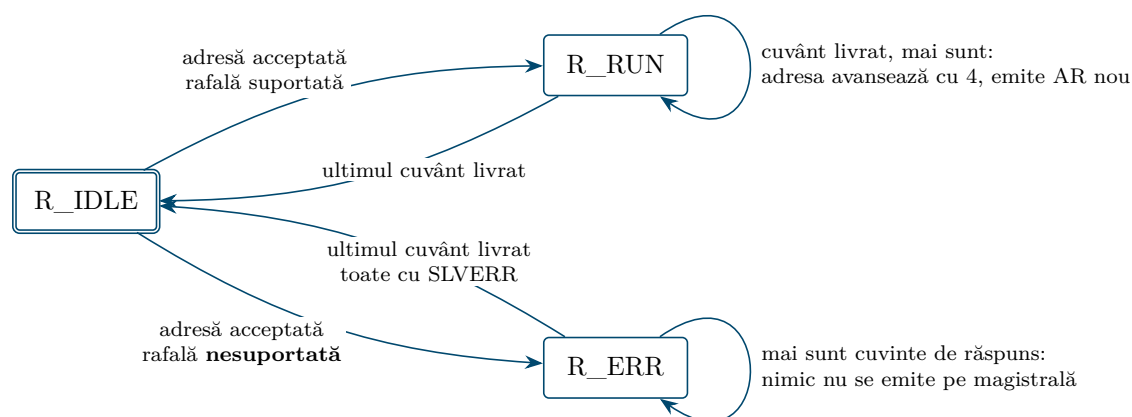
Prima constatare, și cea care a dictat tot restul, a fost că blocurile nu vorbesc același protocol.

Motorul DMA este master **AXI4-Full** și emite rafale **INCR** de opt cuvinte de 32 de biți. Toate slave-urile din sistem — memoriile, memoria duală, perifericele — sunt **AXI4-Lite**, care nu are rafale deloc: o adresă, un cuvânt, un răspuns. Nu există niciun fel de cablare care să le lege.

Am scris deci o punte de protocol, care este slave pe partea dinspre DMA și master pe partea dinspre magistrală. Ea desface fiecare rafală într-o tranzacție AXI4-Lite per cuvânt și reconstruiește ceea ce partea Full așteaptă înapoi: marcajul ultimului cuvânt la citire și *un singur* răspuns pentru toată rafala la scriere.

6.1.1 Calea de citire

Citirile și scrierile sunt independente, așa cum le ține și AXI, iar fiecare parte tratează câte o rafală o dată — atât are masterul DMA în curs simultan.



Suportat: rafale INCR și FIXED, cuvinte de 32 de biți.
Nesuportat: WRAP, transferuri mai înguste de 32 de biți.
Într-o rafală FIXED adresa nu avansează — este singura diferență.

Figura 6.1: Mașina de stări a căii de citire din punte. Starea de eroare există ca să răspundă la o rafală nesuportată *fără* să emită nimic pe magistrală.

Calea de date la citire este o trecere directă, nu o memorare urmată de retransmitere: cuvântul venit de pe partea Lite comandă direct canalul de date al părții Full, iar disponibilitatea masterului condiționează disponibilitatea punții. Am ales așa pentru că menține latența la un ciclu în loc de două, și nu poate nici să reordoneze, nici să piardă date — nu există niciun tampon în care ceva să rămână.

6.1.2 Calea de scriere și răspunsul persistent

Scrierea are o diferență esențială față de citire: AXI4-Full dă unei rafale **exact un** răspuns de scriere, în timp ce puntea primește câte unul de la fiecare dintre cele opt tranzacții Lite. Trebuie deci să le combine.

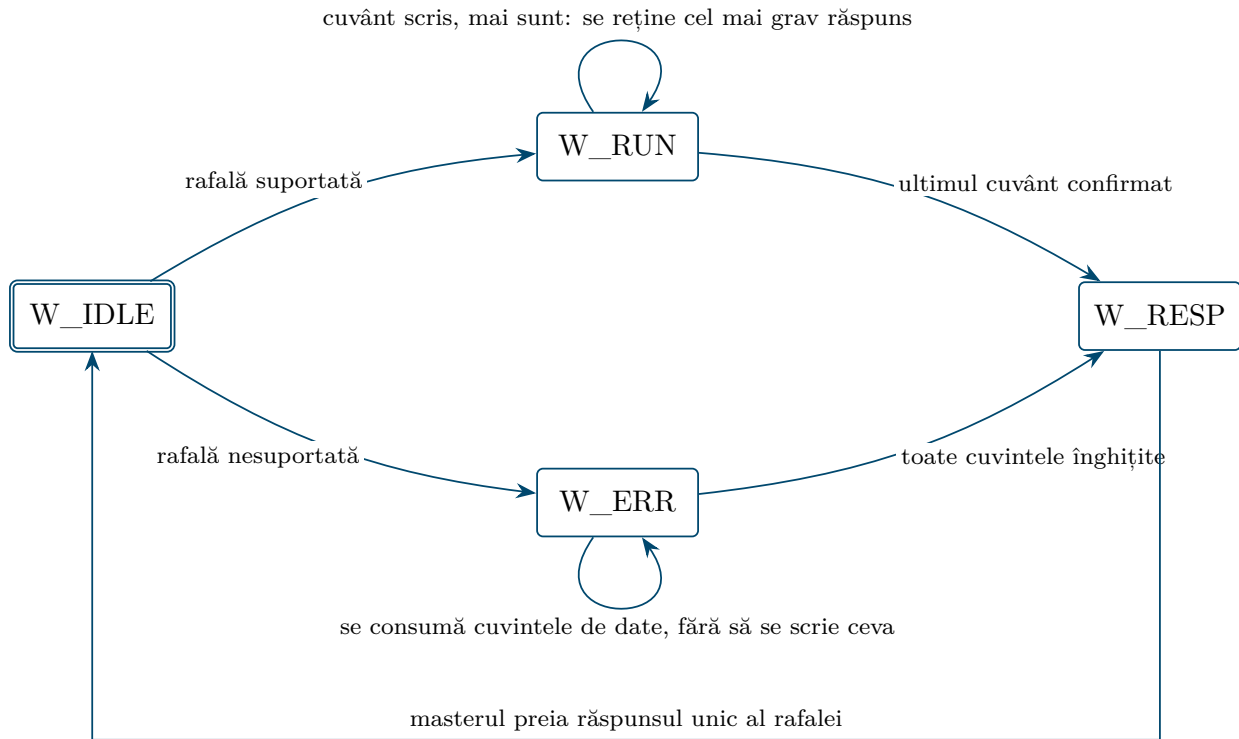


Figura 6.2: Mașina de stări a căii de scriere. Starea finală există exact pentru că protocolul cere un singur răspuns pe rafală: el nu poate fi trimis până nu s-a încheiat ultima tranzacție Lite.

Am luat două decizii aici, și pe amândouă le consider printre cele mai importante din tot proiectul:

- **Răspunsul rafalei este cel mai grav răspuns primit de oricare cuvânt din ea.** Dacă al treilea cuvânt din opt eșuează, iar restul reușesc, rafala raportează eroare. Alternativa — să raportezi ultimul răspuns — ar face ca un transfer parțial eșuat să arate curat, ceea ce este exact tipul de defect care se descoperă abia în producție.
- **O rafală nesuportată nu este tradusă greșit, ci refuzată.** O rafală de tip WRAP sau un transfer mai îngust primește SLVERR pe toată lungimea ei, iar pe magistrală nu se emite absolut nimic. A transforma în tăcere o rafală neînțeleasă în adrese greșite este felul în care o punte corupe memoria într-un mod pe care nimeni nu-l găsește o săptămână.

Puntea răspunde la rafala nesuportată *integral*: îi datorează masterului tot atâtea cuvinte câte a cerut, doar că fiecare vine cu eroare. Dacă ar răspunde mai puține, masterul ar rămâne blocat așteptând restul — iar un master blocat este mult mai greu de diagnosticat decât un transfer care a eșuat curat.

6.2 Decodorul de adrese și bucla combinațională

Decodorul rutează un master către unul dintre N slave-uri, în funcție de adresă, și răspunde el însuși cu DECERR la orice adresă care nu cade în nicio fereastră. Fără acest răspuns propriu, un acces nemapat ar bloca masterul la nesfârșit, așteptând un răspuns care nu vine niciodată — cel mai derutant mod în care poate eșua o magistrală.

Cea mai importantă decizie din decodor este că cele două faze ale unei tranzacții nu sunt rutate după același criteriu.

Faza de adresă și faza de răspuns nu se rutează la fel. Faza de adresă este rutată după adresa vie de pe magistrală, cât timp semnalul ei de validare este sus, astfel încât datele de scriere prezentate odată cu adresa să urmeze același slave. Faza de răspuns este rutată **exclusiv** după selecția memorată la handshake-ul de adresă — niciodată după adresa vie.

Motivul este că altfel se închide o buclă combinațională prin decodor. Unitatea de fetch a instrucțiunilor emite adresa pentru instrucțiunea $N+1$ în *același ciclu* în care primește datele pentru instrucțiunea N . Dacă multiplexorul de răspuns s-ar uita la adresa vie, validitatea datelor citite ar depinde combinațional de validitatea adresei; iar cum validitatea adresei depinde de cuvântul tocmai primit, cele două s-ar închide în buclă, iar simularea s-ar opri cu o eroare de limită de iterații.

Selecția memorată nu este doar o ocolire a buclei, ci este și corectă pe fond: un răspuns nu poate apărea decât după handshake-ul propriei adrese, deci în momentul în care răspunsul poate exista, selecția memorată este întotdeauna cea potrivită.

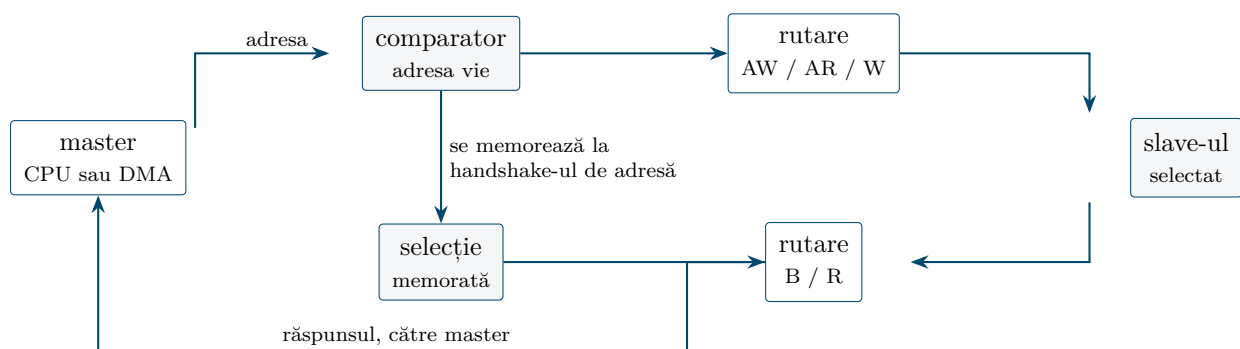


Figura 6.3: Cele două căi de rutare din decodor. Faza de adresă urmează adresa vie; faza de răspuns urmează exclusiv selecția memorată la handshake-ul de adresă.

Un al doilea detaliu, mai mic dar din aceeași familie: comparațiile de adresă sunt condiționate de validitatea adresei. Magistrala de adrese a unui master este nedefinită între tranzații, fiindcă registrele de sarcină utilă nu au reset. O comparație necondiționată ar propaga valori nedefinite în multiplexoarele de disponibilitate, iar valorile nedefinite se propagă tăcut prin tot sistemul.

6.3 Arbitrul

Unde doi masteri trebuie să împartă un slave cu un singur port — în sistemul de față, portul de date al procesorului și DMA-ul, pe memoria de date — am pus un arbitru round-robin.

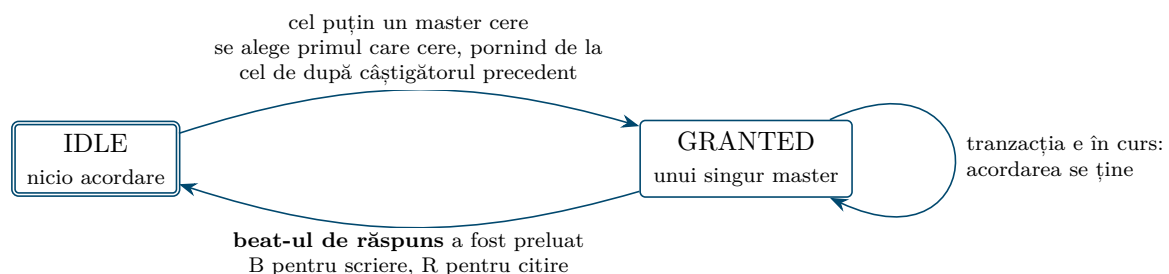


Figura 6.4: Arbitrul. O acordare durează exact o tranzație și este eliberată pe beat-ul de răspuns, nu la handshake-ul de adresă.

Momentul eliberării acordării este singura decizie reală aici, și este ușor de greșit. Dacă acordarea s-ar elibera la handshake-ul de *adresă*, al doilea master ar putea primi slave-ul în timp

ce răspunsul primului este încă pe drum, iar acel răspuns ar ajunge la masterul greșit. Ținând acordarea până la beat-ul de răspuns, o tranzacție este indivizibilă.

Pornirea căutării de la masterul de după ultimul câștigător este ceea ce face arbitrul round-robin în loc de prioritate fixă — și, cum o acordare durează o singură tranzacție AXI4-Lite (care nu are rafale), niciun master nu poate ține slave-ul un timp nelimitat și niciunul nu poate fi înfometat.

Un master care nu are acordarea își vede pur și simplu semnalele de disponibilitate joase, ceea ce protocolul permite pe termen nedefinit. Nu vede niciodată un handshake parțial, pentru că numai semnalele masterului acordat sunt conectate la slave.

6.4 Memoriile și nivelul de vârf

Am scris și o memorie AXI4-Lite sintetizabilă, cu măști pe octet și încărcare de imagine la pornire, folosită pentru memoria de instrucțiuni și cea de date. Rațiunea este că sistemul trebuie să fie un proiect complet, nu unul care se elaborează doar cu un model comportamental atașat. Matricea este ștearsă înainte de încărcarea imaginii, deci nicio valoare nedefinită nu ajunge la procesor.

Memoria are și parametri de latență și de contra-presiune, inerți la valorile implicite. Ei sunt cei care permit regresiei să parcurgă temporizări diferite de magistrală — vezi Secțiunea 7.3.1.

Nivelul de vârf leagă totul: procesorul, motorul DMA, memoria duală, cele două memorii, controlerul de întreruperi, temporizatorul, cele trei decodoare, arbitrul și puntea. Harta de adrese este definită într-un singur loc și dată ca parametru fiecărui decodor, deci nu există nicio adresă scrisă de mână în sistem.

6.5 Delimitarea față de blocurile colegilor

Integrarea respectă o regulă explicită: **nicio schimbare de algoritm în blocul altcuiva**. Nicio mașină de stări, nicio arbitrare și niciun detector de coliziuni din blocurile colegilor nu este modificat de magistrala pe care am scris-o.

Tabela 6.1: Ce atinge integrarea, în funcție de cine deține codul

Cine deține codul	Ce face integrarea
Blocul propriu: procesor, controler de întreruperi, temporizator, harta de adrese	Îl modific direct. Proprietățile pe care regresia le fixează peste ele sunt în Secțiunea 7.6.
Blocurile colegilor: motorul DMA, memoria duală	Nu le ating. Ce este găsit acolo se documentează cu fișier și linie și se trimite proprietarului, care decide. A repara codul altcuiva în ramura de integrare ar face ca versiunea lui și a mea să divergă tăcut, iar testele lui ar continua să ruleze pe cod care nu mai este cel livrat.
Nici numele porturilor lor	Convenția de denumire diferă între blocuri, dar rescrierea listei de porturi a unui bloc îi schimbă interfața și îi invalidează testbench-urile proprii; nivelul de vârf se conectează la numele așa cum sunt.

6.6 Traseul complet, pas cu pas

Ce se întâmplă efectiv când software-ul cere un transfer — scenariul din tema stagiului, urmărit prin tot ce am scris:

1. Programul construiește un descriptor în memoria de date: sursă, destinație, lungime, control.

2. Scrie adresa descriptorului și bitul de pornire în registrele DMA-ului, pe portul de date al procesorului, prin decodorul de date, ieșirea 4.
3. Activează sursa în controlerul de întreruperi și bitul din registrul de activare al procesorului, apoi execută instrucțiunea de așteptare; magistrala de instrucțiuni tace.
4. Canalul DMA își aduce descriptorul și emite rafale. Fiecare rafală intră în punte, care o desface în opt tranzacții AXI4-Lite.
5. Fiecare tranzacție intră în decodorul DMA-ului: cele către memoria de date se așază la coadă în arbitru, alături de cererile procesorului; cele către memoria duală merg direct la portul B.
6. La terminare, canalul își ridică linia de întrerupere; ea trece prin masca DMA-ului, prin cea a controlerului, și ajunge la procesor.
7. Procesorul se trezește, confirmă preluarea, execută rutina de tratare, compară ce a mutat DMA-ul cu ce i-a cerut, și anunță terminarea.

Capitolul 7

Verificarea

Jumătate din titlul temei este „verificare avansată”, și aproape jumătate din timpul stagiului a mers acolo.

7.1 Strategia

Am folosit patru mecanisme diferite, pentru că fiecare vede altceva:

Tabela 7.1: Cele patru mecanisme de verificare și ce vede fiecare

Mecanism	Ce vede, și ce nu vede
Teste direcționate, auto-verificatoare	Verifică scenarii concrete: „scriu asta, aștept atâta, trebuie să citesc aceea”. Fiecare test își compară singur rezultatul și tipărește un verdict, deci un jurnal poate fi validat numărând verdicte, nu citindu-l. Nu vede însă decât scenariile la care m-am gândit.
Aserțiuni	Reguli permanente, verificate în fiecare ciclu al fiecărei simulări în care sunt active. Ele prind încălcări în situații la care nu s-a gândit nimeni — dar numai încălcări de <i>regulă</i> , nu rezultate greșite.
Acoperire funcțională	Arată ce a rămas neexersat. Nu spune că ceva e corect; spune că ceva nu a fost încercat niciodată.
Regresie	Rulează totul dintr-o singură compilare, la fiecare modificare. Rolul ei nu este să găsească defecte noi, ci să împiedice reapariția celor vechi.

Peste ele am adăugat o a cincea metodă, care s-a dovedit cea mai valoroasă: **injectarea deliberată de defecte**, descrisă în Secțiunea 7.5.

7.2 Mediul de verificare

Testbench-urile de bloc verifică fiecare modul separat: unitatea aritmetică, predictorul de branch, registrele de control, cauzele de trap, controlerul de întreruperi cu benzile și imbricarea lui, temporizatorul. Testbench-urile de sistem verifică ansamblul.

Cele de sistem sunt speciale prin faptul că **verificarea o face software-ul, nu testbench-ul**. Testbench-ul furnizează doar un ceas, un reset și o imagine de memorie; programul asamblat construiește descriptorul, programează canalul DMA, adoarme pe instrucțiunea de așteptare, este trezit de întreruperea de terminare și compară ce a mutat DMA-ul cu ce i-a cerut. Este cea mai apropiată formă de verificare de utilizarea reală a sistemului.

Am adăugat și monitoare de protocol pe fiecare magistrală, care urmăresc handshake-urile independent de ce cred masterul și slave-ul că se întâmplă.

7.3 Regresia

Tabela 7.2: Regresia procesorului: 15 rulări dintr-o singură compilare

Rulări	Ce verifică	Observație
1–4	Sistem cu un nucleu	Aceleași teste sub patru temporizări diferite de magistrală.
5	Două nuclee, memorie partajată	Handshake între nuclee.
6–7	Controlerul de întreruperi	Funcțional, apoi colțurile de planificare: masca procesorului, frontul sosit exact în ciclul confirmării, escaladarea, declanșarea cu cheie.
8–10	Reset, registre read-only, stare per sursă	Fiecare bit rezervat și fiecare registru numai-citire, individual.
11	Temporizatorul	Registre, armare, rostogolire.
12	Registre de control și stare	Numai-citire, valori inadmisibile, reset.
13	Cauzele de trap	Câte un test dedicat pentru fiecare cauză.
14	Predictorul de branch	Tabel, stivă de retur, limite, reset.
15	Unitatea aritmetică	Decodarea operațiilor și valorile de la limită.

Tabela 7.3: Regresia sistemului: 24 de rulări dintr-o singură compilare

Rulări	Ce verifică	De ce există separat
1–2	Harta de adrese	Întâi cele două declarații ale hărții, una față de alta; apoi ferestrele în circuit. Verifică o proprietate pe care niciun program corect nu o poate atinge: software-ul corect nu emite niciodată o adresă care nu trebuie să decodeze.
3–4	Puntea de protocol	Singura bucată de logică nouă prin care trece toată calea de date a sistemului, testată izolat: întâi rafalele curate, apoi o eroare pe un beat din interiorul rafalei.
5	Periferele sub contra-presiune	Controlerul, temporizatorul și memoria duală cu răspunsul ținut 1, 3 și 10 cicluri.
6–9	Sistemul complet	Software real pe procesor real, conducând un transfer DMA real, cu procesorul adormit cât timp acesta se desfășoară.
10–13	Sistemul sub stres	Același sistem, cu procesorul lucrând pe magistrală tot timpul. Este grupul care atinge logica de contenție.
14–17	Lungimi de transfer	Cinci lungimi care nu sunt multipli întregi de rafală: 64, 40, 20, 8 și 4 octeți.
18–20	Canalele DMA	O întrerupere mascată la periferic, cele patru canale în arbitrare round-robin, și o rafală care depășește fereastra.
21	Temporizatorul	Armat de un program, cu întreruperea livrată prin controler.
22–24	Controlerul, prin procesor	Toate cele șaisprezece surse pe rând, preempția și imbricarea, și escaladarea pe termen limită.

Grupul de stres și cel de lungimi există fiindcă am *măsurat* că testul de sistem simplu nu le atingeau:

- Testul de sistem simplu **nu a contendat niciodată arbitrul**, nu a atins niciodată ambele porturi ale memoriei duale în același ciclu și nu a produs nicio coliziune reală. Trecea fără să exerseze nimic din toate acestea. Testul de stres le măsoară și **cade dacă nu s-au întâmplat**, deci acoperirea nu poate să se degradeze tăcut.

- Cele două programe de sistem mută 64 și 256 de octeți — multipli exacti ai rafalei de 32 de octeți. Calea rafalei finale scurte era deci inaccesibilă din testele de sistem, iar exact acolo scrie un DMA dincolo de ce i s-a cerut. Testul de lungimi citește direct matricea memoriei și cade atât la un cuvânt livrat scurt, cât și la unul scris dincolo de capăt.

7.3.1 Parcurgerea temporizărilor de magistrală

Fiecare test la nivel de sistem rulează sub patru temporizări de magistrală: nominală, latență fixă mare, și două semințe diferite de contra-presiune aleatoare pe semnalele de disponibilitate.

Motivul este direct: o magistrală care funcționează numai când fiecare slave răspunde într-un ciclu nu a fost testată. Decodorul ține o selecție memorată peste răspuns, arbitrul ține o acordare peste el, iar puntea ține un contor de cuvinte peste el — și toate trei sunt exact genul de stare care se strică doar când slave-ul își ia timp.

Pragurile de acoperire sunt impuse în *toate* cele patru configurații, nu doar în cea nominală. Nu era garantat: latența injectată schimbă planificarea, deci programul ajunge de fiecare dată în alte colțuri. Măsurat, pragurile se țin peste tot — arbitrul este contendat între 161 și 224 de cicluri, și cel puțin o coliziune reală de adresă apare în fiecare configurație.

7.4 Aserțiunile

Stratul de aserțiuni este legat de circuit, nu scris în el, și rulează pe Verilator, fiindcă ediția de ModelSim disponibilă nu compilează aserțiuni. El acoperă trei zone: protocolul AXI4-Lite și AXI4-Full pe fiecare port, invarianții nucleului și ai controlerului de întreruperi, și deciziile proprii ale magistralei:

- ferestrele de adresă sunt disjuncte;
- acordarea arbitrului este one-hot, adică cel mult un master este servit;
- acordarea este ținută până la beat-ul de răspuns;
- un master neacordat nu vede niciun handshake;
- contorul de cuvinte al punții și marcajul de ultim cuvânt sunt de acord;
- confirmarea și terminarea întreruperii sunt impulsuri de un ciclu și nu apar niciodată în același ciclu;
- adâncimea de imbricare se mișcă numai pe aceste două impulsuri și nu trece niciodată de limită.

Stratul rulează pe toate cele cincisprezece testbench-uri ale fluxului Verilator de sistem, plus testul de sistem al procesorului, fără nicio declanșare.

O aserțiune care nu e legată nu verifică nimic. Am verificat explicit că stratul chiar este activ, nu doar prezent: aceeași simulare compilată fără suportul de aserțiuni raportează zero aserțiuni, iar cu el le raportează pe ale sale și indică drumul ierarhic din interiorul circuitului. Verificarea pare paranoică, dar un strat de aserțiuni prezent în surse și nelegat de circuit arată, într-un jurnal de simulare, exact la fel ca unul care funcționează.

7.5 Injectarea deliberată de defecte

Un test care trece poate trece din motive greșite până la proba contrarie. Ca să aflu care este cazul, am injectat șapte defecte, unul câte unul, și am rulat regresia completă pentru fiecare.

Tabela 7.4: Defecte injectate deliberat și testul care le prinde

Defect injectat	Prins de
Prioritatea coliziunii din memoria duală, inversată	testul de stres
Persistența răspunsului de eroare, eliminată din punte	testul de eroare pe un beat din rafală
Fereastra memoriei duale lărgită la de patru ori dimensiunea ei	testul hărții de adrese
Acordarea arbitrului eliberată la handshake-ul de adresă	testul de sistem
Răspunsul decodurului rutat după adresa vie	bucla combinațională oprește simularea
Marcajul de ultim cuvânt mutat cu un cuvânt mai devreme	testul punții, toate testele de sistem, și o aserțiune
Masca de întreruperi ignorată la formarea liniei	testul canalului DMA cu întreruperea mascată

Fiecare dintre cele șapte cade acum în regresie. Două nu cădeau la prima rundă de injectare — persistența răspunsului de eroare și masca de întreruperi — și exact acelea două au dat ultimele două testbench-uri: unul care răspunde cu eroare pe un beat ales din interiorul rafalei, altul care lasă un canal DMA cu întreruperea mascată. Injectarea nu a găsit un defect în circuit; a arătat unde suita nu se uita.

Rezultatul este cel mai util al întregului stagiu, pentru că este singurul care spune ceva despre *suită*, nu despre circuit: un raport de acoperire arată ce nu a fost atins, iar injectarea unui defect arată dacă ceea ce a fost atins era și privit.

7.6 Proprietăți pe care regresia le fixează explicit

Nouă proprietăți ale blocului meu nu se citesc din structura codului și se pierd tăcut dacă cineva le atinge: comportamentul rămâne rezonabil în cazul obișnuit și se rupe într-un colț. Fiecare are un test dedicat în regresie, care cade dacă proprietatea dispăre. Împreună, ele sunt specificația fină a blocului — partea pe care o hartă de registre nu o poate exprima.

Tabela 7.5: Proprietăți fixate de câte un test dedicat

Proprietate garantată	Unde este fixată
Revenirea dintr-un trap știe ce fel de trap închide: câte un bit per nivel deschis, 16 adânc.	testbench-ul de cauze de trap
O sursă mascată de procesor nu este oferită și nu blochează sursele mai puțin urgente din spatele ei.	programul de sistem, la parcurgerea canalelor
Registrul de surse în așteptare arată întregul set în așteptare, nu doar sursa oferită în acel moment.	testbench-ul funcțional al controlerului
Fereastra fiecărui periferic are exact 256 de octeți, cât banca lui de registre, deci nimic nu aliasează deasupra blocului.	testbench-ul hărții de adrese
Escaladarea urcă în ordinea de urgență configurată a benzilor; nu presupune că banda 0 este cea mai urgentă.	testbench-ul de planificare
Un front sosit exact în ciclul confirmării nu se pierde: frontul are prioritate față de confirmare.	testbench-ul de planificare
Declanșarea software cere bitul de cerere și cheia scrise în aceeași tranzacție; cheia singură nu face nimic.	testbench-ul de planificare
Vectorul de trap nu stochează moduri rezervate: un mod rezervat se citește ca mod direct.	testbench-ul de registre read-only
O operație atomică pe un contor de performanță se aplică pe valoarea deja incrementată, deci evenimentul ciclului curent nu se pierde.	testbench-ul de registre read-only

Compunerea celor două măști de întreruperi este singura dintre ele care nu se poate rezolva în interiorul unui singur bloc.

7.6.1 Compunerea celor două măști de întreruperi

O sursă trece prin două măști independente înainte să ajungă la o rutină de tratare: activarea din controlerul de întreruperi și bitul din registrul de activare al procesorului. Felul în care cele două se compun este o decizie de proiectare, nu o consecință automată a faptului că amândouă există.

Nucleul predă masca lui controlerului, iar rezolvitorul sare peste sursele mascate. O sursă mascată rămâne totuși *în așteptare*: apare în registrul de surse în așteptare și contorul ei de termen limită continuă să meargă. Doar că nu este oferită nucleului până când masca se ridică.

Aranjamentul opus — controlerul alege un câștigător fără să știe nimic despre masca procesorului, iar procesorul aplică masca asupra câștigătorului primit — pare echivalent și nu este. O sursă activată în controler dar mascată în procesor ar fi aleasă și niciodată preluată: rezolvitorul ar rămâne parcat pe ea, nimic mai puțin urgent nu ar mai fi oferit vreodată, iar o instrucțiune de așteptare care depinde de o astfel de sursă nu s-ar mai trezi. Cele două măști n-ar compune o mască mai strictă, ci un blocaj.

Proprietatea este fixată de programul de sistem, care lasă o sursă activată în controler și mascată în procesor pe toată durata rulării și verifică dacă sursele din spatele ei sunt totuși servite.

7.7 Rezultate măsurate

Cifrele de mai jos sunt citite din jurnalul regresiei rulate pe versiunea curentă a proiectului, nu din memorie.

Tabela 7.6: Rezultatele regresiei complete

Regresie	Rulări	Verificări	Rezultat
Procesor, controler de întreruperi, temporizator	15	2 561	toate trec
Sistemul complet	24	428	toate trec
Total	39	2 989	0 eșecuri

Toate cele trei blocuri se compilează într-o singură bibliotecă, fără erori și fără avertismente. Analiza statică pe al doilea simulator este curată, iar stratul de aserțiuni trece fără nicio declanșare pe toate testbench-urile de sistem. Acoperirea funcțională măsurată pe nucleu atinge 88 din cele 92 de coșuri și trece pragul impus, fiindcă fiecare coș obligatoriu a fost atins.

Capitolul 8

Rezultate și concluzii

8.1 Ce funcționează

Sistemul face astăzi exact ce cerea tema, verificat automat: software rulând pe procesorul pe care l-am proiectat construiește un descriptor, programează un canal DMA, adoarme, este trezit de întreruperea de terminare trecută prin controlerul de întreruperi pe care l-am proiectat, și verifică octet cu octet ce a fost mutat. Aceleași verificări trec sub patru temporizări diferite de magistrală, inclusiv cu contra-presiune aleatoare, și cu procesorul lucrând pe magistrală în paralel cu transferul.

Contribuția mea, rezumată:

Tabela 8.1: Ce am livrat

Componentă	Conținut
Procesor RV32I	Pipeline pe trei stagii cu trap-uri precise, predictor de branch cu stivă de retur, unitate load/store cu măști pe octet și excepții precise, două porturi master AXI4-Lite, registre de control M-mode, contoare de performanță, instrucțiune de așteptare.
Controler de întreruperi	16 surse hardware și 16 canale software, benzi de prioritate programabile, preemptie cu imbricare până la 16 niveluri, detecție de cereri spurious, termene limită cu escaladare, 56 de registre mapate.
Temporizator	Contor și comparație pe 64 de biți, cu secvență de armare fără declanșare falsă.
Magistrala sistemului	Punte AXI4-Full → AXI4-Lite, decodor de adrese cu răspuns propriu la adrese nemapate, arbitru round-robin, memorie AXI4-Lite sintetizabilă, nivelul de vârf și harta de adrese.
Verificare	39 de rulări de regresie cu 2989 de verificări automate, strat de aserțiuni pe fiecare magistrală, acoperire funcțională cu praguri impuse, injectare deliberată de defecte, integrare continuă.

Tabela 8.2: Scara lucrării, numărată în depozit

	Fișiere	Linii
RTL: procesor, controler de întreruperi, temporizator, magistrală	23	4844
Testbench-uri proprii, de bloc și de sistem	32	9316
Fișiere de aserțiuni	9	—

8.2 Ce am învățat

8.2.1 Despre proiectare

AXI mi-a părut simplu până când a trebuit să scriu un master. Diagrama cu cele cinci canale se învață în zece minute; ce contează de fapt sunt restricțiile, iar pe acelea le-am descoperit una câte una, în simulare: o cerere ridicată nu mai poate fi retrasă, nu se poate presupune nicio ordine între canalul de adresă și cel de date la o scriere, iar o citire pornită nu se poate anula. Ultima are o

consecință pe care nu o anticipasem — unitatea de fetch a instrucțiunilor are un mecanism întreg, indicatorul de aruncare, care există numai pentru că răspunsul unei citiri deja emise trebuie primit și apoi ignorat.

Al doilea lucru pe care îl duc mai departe ține de cum reprezintă starea. Urmărirea nivelurilor de trap părea, la prima vedere, o treabă pentru un singur bit — „suntem sau nu într-o întrerupere”. Cu un bit, comportamentul este corect pentru un nivel deschis, adică exact cazul pe care îl atinge un test scris fără intenție adversarială, și greșit pentru două. De atunci, pentru fiecare indicator de stare pe care îl adaug, mă întreb câte niveluri poate avea simultan lucrul pe care îl descrie.

Cât despre erori, cele două decizii din puncte — răspunsul cel mai grav câștigă, iar o rafală neînțeleasă este refuzată în loc să fie tradusă — vin din aceeași idee: un sistem care raportează un eșec se poate depăna, iar unul care îl ascunde, nu.

8.2.2 Despre verificare

Cel mai mult mi-a schimbat felul de a gândi momentul în care am măsurat acoperirea pe un test care trecea. Testul de sistem trece impecabil și, luat singur, nu atinge arbitru sub contenție, nu atinge ambele porturi ale memoriei duale în același ciclu și nu produce nicio coliziune — adică nu exersează niciuna dintre situațiile pentru care logica aceea este scrisă. De aceea există testul de stres alături de el.

Mi-au rămas de acolo două obiceiuri. Pragurile de acoperire se impun, nu se tipăresc: un test care numără contenții și nu cade când ele lipsesc nu verifică nimic. Și un test care trece merită tratat ca suspect până când i se arată că poate să cadă — injectarea deliberată de defecte este cel mai ieftin mod de a afla asta, și mi-a spus mai multe despre calitatea suitei mele decât orice raport de acoperire.

8.2.3 Despre lucrul în echipă și documentație

Trei blocuri dezvoltate în paralel de trei oameni nu se integrează dacă interfața nu este fixată înainte de a începe. Cele trei înțelegeri din prima săptămână — un singur protocol, un singur ceas, întreruperile prin controler — sunt motivul pentru care integrarea a fost o problemă de zile, nu de săptămâni.

Regula pe care nu o anticipasem este că un defect din blocul altcuiva se raportează, nu se repară. Am găsit mai multe probleme în blocurile colegilor și le-am documentat cu fișier și linie, dar le-au corectat ei, pe ramurile lor. Dacă le-aș fi corectat eu în ramura de integrare, versiunea lor și a mea ar fi divergat tăcut, iar testele lor ar fi continuat să ruleze pe cod care nu mai era cel livrat.

Tot de aici vine și ce am învățat despre documentație. Am scris-o în paralel cu codul și, scriind-o, am observat că unele afirmații descriu intenția de proiectare, nu comportamentul implementat. Diferența nu este cosmetică: o documentație care descrie intenția îl trimite pe următorul integrator într-o presupunere greșită.

8.3 Concluzie

Stagiul a acoperit până acum un ciclu complet de proiectare digitală: de la citirea unei specificații și transformarea ei într-o arhitectură, prin implementare RTL și verificare, până la integrarea cu blocuri scrise de altcineva și documentarea rezultatului.

Partea cea mai formativă nu a fost scrierea procesorului, deși a fost cea mai mare ca volum, ci integrarea — locul unde trei bucăți de logică, fiecare corectă în izolare, trebuie să se potrivească — și revizuirea propriei lucrări pornind de la premisa că greșelile sunt acolo. De acolo îmi rămâne o observație pe care nu o aveam la început: un comportament greșit, dacă are documentată o cale de a-l ocoli, arată din interior exact ca o decizie de proiectare. Singurul mod de a le deosebi este să ceri, pentru fiecare, testul care ar cădea dacă proprietatea s-ar pierde.

Anexa A

Tehnologii, instrumente și protocoale

A.1 Limbaje și instrumente

Am lucrat exclusiv cu instrumente folosite industrial pentru proiectare digitală. Alegerile nu au fost ale mele — tema le-a fixat — dar motivul pentru care fiecare există în flux merită enunțat, pentru că fluxul întreg este unul dintre lucrurile pe care le-am învățat în stagiu.

Tabela A.1: Instrumentele folosite și rolul fiecăruia în flux

Instrument	Rol
Verilog-2005	Limbaajul de descriere hardware în care este scris tot circuitul. Am folosit strict subsetul sintetizabil: fără construcții comportamentale în calea de date, un singur bloc always pentru fiecare registru, reset asincron pe fiecare bistabil.
SystemVerilog Assertions	Aserțiuni care descriu reguli permanente ale protocolului — de exemplu „VALID nu poate cădea până nu vine READY”. Spre deosebire de un test, o aserțiune verifică regula în <i>fiecare</i> ciclu al <i>fiecărei</i> simulări în care este activă.
ModelSim și Verilator	Simulatorul principal, pe care rulează regresia completă, și al doilea, folosit pentru două lucruri pe care primul nu le face în ediția disponibilă: analiză statică strictă și compilarea aserțiunilor. Faptul că aceeași sursă trece pe două simulatoare diferite este o verificare în sine — fiecare tolerează alte scăpări.
Asamblor propriu, scris în Python	Programele de test rulează pe procesorul propriu, deci trebuie asamblate. Am scris un asamblor RV32I care produce imaginea de memorie și, în plus, un fișier cu adresele etichetelor, ca testbench-ul să poată verifica unde a ajuns execuția fără să conțină adrese scrise de mână.
Makefile	Un singur punct de intrare pentru fiecare flux: asamblare, regresie pe ModelSim, analiză statică și aserțiuni pe Verilator. O comandă, nu o secvență de pași ținută minte.
Git	Model pe ramuri: o ramură per bloc, integrarea pe ramura principală, ramurile de bloc niciodată rescrise.
Integrare continuă	Fluxul de analiză statică, aserțiuni și acoperire rulează automat la fiecare modificare împinsă în depozit, pe Verilator, care este liber și poate rula într-un mediu automat.
LaTeX	Documentația tehnică și acest raport. Diagramele sunt desenate în TikZ, direct în sursa documentului, ca să poată fi corectate odată cu circuitul.

A.2 Magistrala AXI4-Lite

Toate interfețele din sistem, cu o singură excepție, sunt AXI4-Lite. Este subsetul simplu al standardului AMBA AXI4: un transfer, o adresă, un cuvânt de date. Nu are rafale, nu are identificatori de tranzacție și nu permite reordonarea răspunsurilor.

A.2.1 Canale

Un transfer este purtat pe unul dintre cele cinci canale independente. Fiecare are handshake-ul lui, și niciunul nu blochează alt canal.

Tabela A.2: Canalele AXI4-Lite și conținutul fiecăruia

Canal	Direcție	Transportă
AW	master → slave	Adresa de scriere (AWADDR) și tipul de protecție (AWPROT).
W	master → slave	Datele de scris (WDATA) și măștile pe octet (WSTRB).
B	slave → master	Răspunsul la scriere (BRESP).
AR	master → slave	Adresa de citire (ARADDR) și tipul de protecție (ARPROT).
R	slave → master	Datele citite (RDATA) și răspunsul la citire (RRESP).

O citire folosește AR, apoi R. O scriere folosește AW și W — care sunt canale separate și pot fi acceptate în orice ordine — și se încheie cu B. Nu există niciun semnal „scriere terminată” în afară de răspunsul B, și acesta este motivul pentru care fiecare cale de scriere pe care am scris-o urmărește AW și W independent unul de altul.

A.2.2 Handshake-ul

Toate canalele funcționează la fel. Sursa ridică VALID când are ceva de transferat; destinația ridică READY când poate primi. **Transferul are loc pe frontul de ceas în care ambele sunt sus.** Oricare dintre ele poate veni primul.

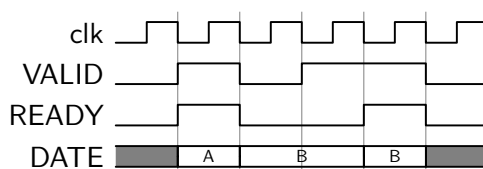


Figura A.1: Două transferuri pe același canal. Primul (sarcina utilă A) se încheie imediat, pentru că ambele părți sunt gata în același ciclu. Al doilea (sarcina B) este ținut de sursă încă două cicluri, cât timp destinația nu este gata; sarcina utilă nu se schimbă pe durata așteptării.

Din această regulă decurg două obligații, și amândouă sunt verificate automat pe fiecare magistrală din sistem, de aserțiunile descrise în Secțiunea 7.4:

1. Odată ridicat, VALID nu poate cădea până nu a fost acceptat de READY. Un master care își retrace cererea pentru că s-a răzgândit încalcă protocolul.
2. Cât timp VALID este sus și neacceptat, sarcina utilă nu se poate schimba. Altfel destinația nu ar ști ce a primit.

A.2.3 Codurile de răspuns

Tabela A.3: Codurile purtate de RRESP și BRESP

Cod	Nume	Înțeles în acest sistem
00	OKAY	Transferul a reușit.
10	SLVERR	Adresa a ajuns la un slave, dar slave-ul a refuzat cererea: acces nealiniat, rafală nesuportată, sau coliziune pierdută pe memoria dual-port.
11	DECERR	Adresa nu aparține niciunui slave. Îl produce decodorul, nu un periferic.

Distincția dintre ultimele două contează practic: SLVERR înseamnă „am ajuns unde trebuia, dar cererea era greșită”, iar DECERR înseamnă „nu există nimic la adresa aceea”. În sistemul de față ambele produc o excepție precisă în procesor, dar cu cauze distincte, deci un program poate spune care dintre ele s-a întâmplat.

A.3 Ce adaugă AXI4-Full

Portul de master al DMA-ului este AXI4-Full. Diferența relevantă pentru acest proiect este una singură, dar are consecințe pe tot lanțul.

Tabela A.4: Diferențele dintre AXI4-Full și AXI4-Lite relevante aici

	AXI4-Lite	AXI4-Full
Rafale	Nu există. O adresă, un cuvânt.	AWLEN/ARLEN cer până la 256 de cuvinte pentru o singură adresă.
Ultimul cuvânt	Nu are sens.	WLAST și RLAST marchează sfârșitul rafalei.
Răspuns la scriere	Unul per transfer.	Unul singur pentru toată rafala.
Lățimea transferului	Fixă, 32 de biți.	AWSIZE/ARSIZE permit și transferuri mai înguste.
Tipul rafalei	—	AWBURST/ARBURST: INCR (adresa crește), FIXED (adresa rămâne pe loc), WRAP (adresa se învârtă într-un bloc aliniat).

Motorul DMA emite rafale INCR de opt cuvinte de 32 de biți, adică 32 de octeți deodată. Toate slave-urile din sistem sunt AXI4-Lite. Nimic nu le poate lega direct, și acesta este motivul pentru care am scris puntea din Secțiunea 6.1.

A.4 Arhitectura RISC-V RV32I

Procesorul implementează setul de instrucțiuni de bază RV32I: 32 de registre generale de 32 de biți, instrucțiuni aritmetice și logice, branch-uri condiționate și jump-uri necondiționate, încărcări și stocări, și instrucțiunile de sistem. Fără extensii — fără înmulțire hardware, fără virgulă mobilă, fără instrucțiuni comprimate.

Peste setul de bază este implementat nivelul privilegiat *machine mode*, în măsura cerută de întreruperi și de excepții precise: registrele de control și stare, vectorul de trap, adresa de retur, cauza și valoarea asociată. Exact subsetul de care are nevoie un sistem cu întreruperi.

Anexa B

Termeni și abrevieri

Termen	Semnificație
AMBA AXI4	Familia de protocoale de magistrală ARM folosită în tot sistemul.
AXI4-Lite	Subsetul fără rafale: o adresă, un cuvânt, un răspuns.
AXI4-Full	Varianta completă, cu rafale de până la 256 de cuvinte.
RV32I	Setul de instrucțiuni de bază RISC-V pe 32 de biți, cu numere întregi.
M-mode	Nivelul privilegiat „mașină” din specificația RISC-V, singurul implementat aici.
RTL	<i>Register Transfer Level</i> — descrierea circuitului în termeni de registre și logică între ele.
Pipeline	Suprapunerea execuției mai multor instrucțiuni în stagii diferite.
Hazard	Situație în care o instrucțiune nu poate continua fără un rezultat încă indisponibil.
Trap	Termenul RISC-V care acoperă atât excepțiile sincrone, cât și întreruperile.
Trap precis	Proprietatea că, la apariția unui trap, nicio instrucțiune nu este pe jumătate executată.
Preemptie	Întreruperea unei rutine de tratare de către o sursă mai urgentă.
Imbricare	Suprapunerea mai multor rutine de tratare active simultan.
Cerere spurious	Cerere care dispare între oferta controlerului și confirmarea procesorului.
DMA	<i>Direct Memory Access</i> — transfer de date fără intervenția procesorului.
Descriptor	Structura din memorie care descrie un transfer: sursă, destinație, lungime, control.
Scatter-gather	Transfer descris printr-un lanț de descriptori, pentru zone de memorie necontigue.
SRAM dual-port	Memorie cu două porturi de acces fizic independente.
SVA	<i>SystemVerilog Assertions</i> — aserțiuni verificate în fiecare ciclu de simulare.
Acoperire funcțională	Măsura a ceea ce a fost efectiv exersat de teste.
Regresie	Rularea automată a întregii suite de teste la fiecare modificare.
Beat	Un cuvânt dintr-o rafală AXI, cu handshake-ul lui propriu; o rafală de opt beat-uri transferă opt cuvinte.
Bubble	Poziție goală care avansează prin pipeline în locul unei instrucțiuni, marcată printr-un bit de validitate zero.
Testbench	Modulul de simulare care înconjoară circuitul, îi dă stimuli și îi verifică răspunsurile.
Round-robin	Politică de arbitrare care servește cererile pe rând, fără să favorizeze pe nimeni.
Contra-presiune	Întârzierea deliberată a semnalelor de disponibilitate, pentru a testa comportamentul sub magistrală lentă.

Bibliografie

- [1] RISC-V Foundation, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, versiunea 20191213.
- [2] RISC-V Foundation, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, versiunea 20211203.
- [3] ARM Limited, *AMBA AXI and ACE Protocol Specification*, IHI 0022E.
- [4] IEEE, *Standard for Verilog Hardware Description Language*, IEEE Std 1364–2005.
- [5] IEEE, *Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800–2017.